

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a52a7d189c652e8cf.jsonl`  
- SHA-256: `358eabf684860aaa552d5fd67802bef87d63239a4050c72ec32830b329419a4f`  
- Source modified: `2026-07-08T06:14:14+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:04:07.733Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

Your review dimension: test-gaps.

Review the #521 diff's TESTS for gaps that hide real failure modes. The mocks: a fake CloudRunJobClient + local storage. What could pass the mocked tests but break in prod (real GCS, real ffmpeg, real argv round-trip, the worker argparse rejecting an arg, the spec JSON schema mismatch, watermark localize on the worker)? Report only gaps where a plausible real bug would

go uncaught.

2. assistant (2026-07-08T06:04:13.012Z)

I'll start by orienting myself. The diff/source paths came through as `undefined`, so I need to locate the actual PR #525 changes in the repo. Let me pull and inspect the git history.

3. user (2026-07-08T06:04:15.476Z)

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (\$9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)
bd5647b fix(worker): surface detector failure/timeout as failed, not silent 0-rally success (#503)

4. user (2026-07-08T06:04:16.963Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)
63d65f9 fix(cloud-serving): serve large reels via a gs:// signed URL (#463) (#469)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
b09a1d4 serving: re-queue interrupted compile jobs on restart (#329 reliability)
739d1f4 fix: Q2 motion return type + D11 compiler codec docstring (#362)
2c1028a docs(guide): add CREATING_REELS.md ? end-user install?reel walkthrough (#343)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)
fa2342c feat(nightly): E2E engine reel-count regression on a GCP GPU VM (emailed) + cost (#315)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
be0c434 feat(pkg): P1a ? net-new console/wizard endpoints: /api/version, /api/capabilities, /api/videos, /api/reels (#300)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)
e6110ce feat(api): GET /api/health + return download_url from /api/compile (M1/P6) (#268)
d8c5aa0 docs(deploy): report polish ? highlight reel smoke-test result + VM-deleted final state (#262)
3f5a56c fix(gemini): surface failed chunks loudly (no more silent partial reels) (#240)
a3b4954 feat(runner): --min-rally-duration flag (surface the long-rally reel filter on the CLI) (#238)
20f8aa3 feat(eval): P7 long-rally (>5s) quality lens + reel knob (both default-OFF) (#220)
18883b5 refactor(stitcher): extract per-segment trim into _trim_one_segment (#224)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels (#205)
4942d51 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation (#182)
562308b fix(stitcher): merge overlapping/duplicate segments so a reel never stitches the same

rally twice (#180)
 428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree; person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan now accurate. M0 item 3 placement remains correct and even stronger.
 318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys warning, get shim, roundtrip + living doc update for T2 post #157
 8836dfe chore: scripts/ for run-drivers + CODE_MAP \$0 file-placement guidelines (#138)
 6de70aa fix(stitcher): escape drive-letter colon in ffmpeg filtergraph paths (#108)
 3461898 feat(api): enforce stitching.min_shot_count on /api/compile (#107)
 112e75d feat(stitcher): bundle Apache-2.0 default watermark font + log fallback reason
 82e479f Merge pull request #102 from avidullu:fix/stitching-padding-config
 30e4899 fix(api): thread config stitching paddings into live VideoCompiler sites
 10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)

5. assistant (2026-07-08T06:04:21.522Z)

The PR is commit `dac6736` (squash-merge of #521/#525). Let me get the full diff and identify the changed files.

6. user (2026-07-08T06:04:23.726Z)

commit dac6736cb23a681135b707d768014820b239a981
 Author: Avi <avi.dullu@gmail.com>
 Date: Wed Jul 8 11:29:33 2026 +0530

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, \$9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
 Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```

backend/compute/__init__.py | 2 +
backend/compute/base.py | 34 +++
backend/compute/cloud_run.py | 100 +++++-
backend/config/__init__.py | 16 ++
backend/config/models.py | 52 ++++
backend/config/placement.py | 59 ++++
backend/config/presets.py | 10 +-
backend/orchestration.py | 322 ++++++-----
backend/pipeline/stitcher/compiler.py | 94 +++++-
backend/worker/__main__.py | 139 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 169 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 1 +
tests/test_api_cloud_dispatch.py | 70 +++++
tests/test_compile_dispatch.py | 204 ++++++
tests/test_compiler.py | 64 +++++
tests/test_compute_backend.py | 111 ++++++
tests/test_config_deployment.py | 5 +
tests/test_phase_placement.py | 138 ++++++
tests/test_worker.py | 175 ++++++
19 files changed, 1710 insertions(+), 55 deletions(-)

```

7. assistant (2026-07-08T06:04:28.336Z)

Let me get the full diff and read the key source + test files.

8. user (2026-07-08T06:04:32.708Z)

2207 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\scratchpad\pr525.diff

9. user (2026-07-08T06:04:36.121Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1054 of 2208 total (44482 tokens, cap 25000). Call Read with offset=1055 limit=1054 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      commit dac6736cb23a681135b707d768014820b239a981
2      Author: Avi <avi.dullu@gmail.com>
3      Date:   Wed Jul 8 11:29:33 2026 +0530
4
5      perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
6
7      The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the
8      2026-07-08 live CUJ (rev rally-control-plane-00019, S9 of the validation-latency
9      runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source
10     (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode
11     with no NVENC). Validation (183s) also runs there; both serialize on
12     _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a
13     handful of concurrent users.
14
15     - Config-driven placement: deployment.phase_placement {validation|segment|compile}
16     ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ?
17     today's placement, byte-identical. A config-load guard rejects the "nobody
18     validates" footgun (validation on L4 without a cloud segment).
19     - compile ? L4: a kind='compile' Cloud Run dispatch mirroring
maybe_dispatch_remote.
20     The control-plane resolves the reel's segments, stages a compile-spec (resolved
21     time-pairs + stitching config + source uri) to the bucket, dispatches
22     `worker compile` on the L4, bucket-polls a done-marker, and returns the same
23     {status, filepath, download_url} shape the SPA expects. The worker stitches via
the
24     SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.

```

```

25     - NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default
26       h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to
27       libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-
knob
28       now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
29     - validation delegation: phase_placement.validation moves the validator suite off
the
30       control-plane via the existing --skip-validation lever (#513), guarded so an
31       in-process run always validates.
32     - cloud-serving preset places compile on the L4.
33
34     Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
35     Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to
36     test_compute_backend.py (run_compile), test_worker.py (cmd_compile),
37     test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation
38     delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.
39
40     Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>
41
42     diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
43     index 482f3c9..7f8663f 100644
44     --- a/backend/compute/__init__.py
45     +++ b/backend/compute/__init__.py
46     @@ -14,6 +14,7 @@ import os
47     from typing import Any, Optional
48
49     from backend.compute.base import (
50     +     CompileJobSpec,
51     +     ComputeBackend,
52     +     ComputeJobSpec,
53     +     ComputeResult,
54     @@ -24,6 +25,7 @@ from backend.infrastructure.execution_policy import DEFAULT_POLICY,
ExecutionPol
55     __all__ = [
56     +     "ComputeBackend",
57     +     "ComputeJobSpec",
58     +     "CompileJobSpec",
59     +     "ComputeResult",
60     +     "RemoteExecutionFailed",
61     +     "get_compute_backend",
62     diff --git a/backend/compute/base.py b/backend/compute/base.py
63     index 3ad76fc..f7afb40 100644
64     --- a/backend/compute/base.py
65     +++ b/backend/compute/base.py
66     @@ -51,6 +51,24 @@ class ComputeJobSpec:
67     +     skip_validation: bool = False
68
69
70     +@dataclass(frozen=True)
71     +class CompileJobSpec:
72     +     """One reel-COMPILE job dispatched to the worker (#521). The heavy input ? the
resolved
73     +     ``(start, end)`` time-pairs plus the control-plane's stitching/watermark config and
the source
74     +     URI ? is staged to ``spec_uri`` (a small JSON in the bucket) rather than crammed
onto argv, so
75     +     the worker needs no control-plane DB: it reads the spec, fetches the source,
stitches on the L4
76     +     (NVENC), uploads the reel to ``<out_uri>/highlights/<video_id>/<output_name>``, and
writes a
77     +     done-marker for the dispatcher's bucket-poll."""
78     +
79     +     out_uri: str # output ROOT (gs://? bucket); highlights/<video_id>/<name> is
written under it
80     +     spec_uri: str # staged compile-spec JSON (time-pairs + stitching + source uri +

```

```

video_id)
81 + video_id: str # the reel's video_id (deterministic reel path + marker)
82 + output_name: str # traversal-safe reel filename
83 + # Raw worker ``--set k=v`` overrides (storage.output_root/backend +
stitching.encoder). A tuple
84 + # (not list/dict) so the spec stays frozen/hashable.
85 + config_overrides: tuple[str, ...] = ()
86 +
87 +
88 @dataclass(frozen=True)
89 class ComputeResult:
90     """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
91 @@ -85,3 +103,19 @@ class ComputeBackend(ABC):
92     ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend
is
93     still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
94     long GPU run doesn't look frozen. Implementations without a wait loop may
ignore it."""
95 +
96 + def run_compile(
97 +     self,
98 +     spec: "CompileJobSpec",
99 +     *,
100 +     on_wait: "Callable[[float], None] | None" = None,
101 + ) -> ComputeResult:
102 +     """Run a reel-COMPILE ``spec`` to completion and return its ``ComputeResult``
(#521).
103 +
104 +     Non-abstract with a NotImplementedError default: only backends that can offload
the ffmpeg
105 +     stitch (``CloudRunCompute``) override it ? an in-process/other backend never
dispatches a
106 +     compile. The returned ``ComputeResult.returncode`` is the worker's rc (0 ok /
non-zero =
107 +     stitch failure) and ``report`` is the worker's done-marker (carries the reel's
download_url)."""
108 +     raise NotImplementedError(
109 +         f"{type(self).__name__} does not support remote compile dispatch"
110 +     )
111 diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
112 index a4a0743..8df4246 100644
113 --- a/backend/compute/cloud_run.py
114 +++ b/backend/compute/cloud_run.py
115 @@ -37,6 +37,7 @@ from abc import ABC, abstractmethod
116 from typing import Any, Callable, List, Optional
117
118 from backend.compute.base import (
119 +     CompileJobSpec,
120     ComputeBackend,
121     ComputeJobSpec,
122     ComputeResult,
123 @@ -278,21 +279,7 @@ class CloudRunCompute(ComputeBackend):
124     done_uri,
125 )
126
127 - try:
128 -     marker = poll_until(
129 -         lambda: self._poll_check(done_uri, str(execution or "")),
130 -         self._policy,
131 -         label="cloud-run-infer",
132 -         logger=LOG,
133 -         # Live heartbeat (#482): each still-waiting tick reaches the caller so
134 -         # job.progress moves during the whole GPU run instead of freezing.

```

```

135     -         on_tick=on_wait,
136     -     )
137     - except PolicyTimeout as timeout_exc:
138     -     # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
139     -     # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
140     -     # clearly-labelled failure instead of the ~65-min opaque timeout this
branch handles.
141     -     marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
142     +     marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
143     video_id = str(marker.get("video_id", ""))
144     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
145     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
146     @@ -317,6 +304,89 @@ class CloudRunCompute(ComputeBackend):
147     execution=str(execution or ""),
148     )
149
150     + def run_compile(
151     +     self,
152     +     spec: "CompileJobSpec",
153     +     *,
154     +     on_wait: "Callable[[float], None] | None" = None,
155     + ) -> ComputeResult:
156     +     """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
157     +     bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
158     +     dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
159     +     rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
160     +     ~65-min opaque hang."""
161     +     out_root = spec.out_uri.rstrip("/")
162     +     token = self._token or uuid.uuid4().hex
163     +     # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
164     +     # bucket can never collide on a marker.
165     +     done_uri = f"{out_root}/_compile/{token}.done"
166     +     argv: List[str] = [
167     +         "compile",
168     +         "--out-uri",
169     +         spec.out_uri,
170     +         "--spec-uri",
171     +         spec.spec_uri,
172     +         "--done-uri",
173     +         done_uri,
174     +     ]
175     +     # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
176     +     # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
177     +     for kv in spec.config_overrides:
178     +         argv += ["--set", kv]
179     +
180     +     execution = self._client.run_job(
181     +         self._job_name, argv, region=self._region, project=self._project
182     +     )
183     +     LOG.info(
184     +         "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
185     +         self._job_name,
186     +         execution,

```

```

187     +         done_uri,
188     +     )
189     +     marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
190     +     video_id = str(marker.get("video_id", "")) or spec.video_id
191     +     rc = int(marker.get("returncode", 1))
192     +     LOG.info(
193     +         "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
194     +         self._job_name,
195     +         video_id,
196     +         rc,
197     +     )
198     +     # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
199     +     # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
200     +     # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
201     +     return ComputeResult(
202     +         returncode=rc,
203     +         outputs_uri=str(marker.get("reel_uri", "")) or "",
204     +         video_id=video_id,
205     +         report=marker,
206     +         execution=str(execution or ""),
207     +     )
208     +
209     +     def _await_marker(
210     +         self,
211     +         done_uri: str,
212     +         execution: object,
213     +         on_wait: "Callable[[float], None] | None",
214     +         *,
215     +         label: str,
216     +     ) -> dict:
217     +         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer``
and
218     +         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting
tick reaches the
219     +         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
220     +         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
221     +         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
222     +         try:
223     +             return poll_until(
224     +                 lambda: self._poll_check(done_uri, str(execution or "")),
225     +                 self._policy,
226     +                 label=label,
227     +                 logger=LOG,
228     +                 on_tick=on_wait,
229     +             )
230     +         except PolicyTimeout as timeout_exc:
231     +             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
232     +
233     +     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
234     +         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
235
236     diff --git a/backend/config/__init__.py b/backend/config/__init__.py
237     index 3f722c9..14cf483 100644
238     --- a/backend/config/__init__.py
239     +++ b/backend/config/__init__.py
240     @@ -19,10 +19,19 @@ from .models import ( # re-export common ones
241         Config,
242         DeploymentConfig,

```



```

243     IndexingConfig,
244     + PhasePlacementConfig,
245     StitchingConfig,
246     StorageConfig,
247     ValidationConfig,
248 )
249 +from .placement import (
250     + PHASE_COMPILE,
251     + PHASE_SEGMENT,
252     + PHASE_VALIDATION,
253     + PLACE_CLOUD_L4,
254     + PLACE_CONTROL_PLANE,
255     + resolve_phase_placement,
256 +)
257     from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
258     from .startup import (
259         StartupCheck,
260 @@ -52,9 +61,16 @@ __all__ = [
261     "Config",
262     "DeploymentConfig",
263     "IndexingConfig",
264     + "PhasePlacementConfig",
265     "StitchingConfig",
266     "StorageConfig",
267     "ValidationConfig",
268     + "resolve_phase_placement",
269     + "PHASE_VALIDATION",
270     + "PHASE_SEGMENT",
271     + "PHASE_COMPILE",
272     + "PLACE_CONTROL_PLANE",
273     + "PLACE_CLOUD_L4",
274     "PROFILE_PRESETS",
275     "suggest_profile",
276     "probe_cuda_available",
277     diff --git a/backend/config/models.py b/backend/config/models.py
278     index a7c997c..ce8e98a 100644
279     --- a/backend/config/models.py
280     +++ b/backend/config/models.py
281     @@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
282         # background-court fragments), so this keeps the reel to the eye-catching long
283         rallies.
284         # OFF by default ? the detector output is unchanged, only which rallies reach the
285         reel.
286         min_rally_duration: float = Field(default=0.0, ge=0.0)
287     + # Video encoder for the per-clip trim re-encode (the branding watermark forces a
288     + re-encode; the
289     + # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
290     + ``h264_nvenc`` /
291     + # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-
292     + encode that
293     + # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested
294     + encoder ONCE and
295     + # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never
296     + breaks. The
297     + # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for
298     + the L4 run;
299     + # the control-plane's own local stitch keeps libx264.
300     + encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
301     + watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
302
303     @@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
304         gdrive_api: dict[str, Any] = Field(default_factory=dict)

```

```

300     +class PhasePlacementConfig(BaseModel):
301     +     """Per-phase machine placement (#521): move validation / segment(detect) /
compile(stitch)
302     +     between the control-plane and the L4 Cloud Run worker **by config, no code
change**.
303     +
304     +     Each field is ``None`` (? the legacy default for that phase ? see
305     +     ``backend/config/placement.py``) or an explicit ``"control_plane"`` /
``"cloud_run_l4"``.
306     +     ``control_plane`` runs the phase in-process on the CPU-only control-plane;
``cloud_run_l4``
307     +     dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521
behaviour.
308     +     Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read
these directly."""
309     +
310     +     validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
311     +     segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
312     +     compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
313     +
314     +
315     class DeploymentConfig(BaseModel):
316     +     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
317     +     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
S4.3).
318     +     @@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
319     +     +     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
320     +     +     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
321     +     +     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
322     +     +     # Per-phase machine placement (#521). Unset ? today's placement (validation +
compile on the
323     +     +     # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
324     +     +     # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read
the sub-fields
325     +     +     # directly ? the resolver folds in the legacy compute_target default).
326     +     +     phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
327     +     +     # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for
the L4 stitch
328     +     +     # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
329     +     +     # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is
placed on the L4;
330     +     +     # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264
by default).
331     +     +     cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
332     +
333     +     @model_validator(mode="after")
334     +     def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
335     +     +     # A clip can only be validated ON the worker if the worker also DETECTS it
there ? the worker
336     +     +     # is only launched for the segment(detect) phase. So validation=cloud_run_l4
with segment on
337     +     +     # the control-plane would leave NOBODY validating (the control-plane skips, and
no worker runs)
338     +     +     # ? reject that footgun at config load rather than ship a silently-unvalidated
clip.
339     +     +     if self.phase_placement.validation == "cloud_run_l4":
340     +     +         seg = self.phase_placement.segment or (
341     +     +             "cloud_run_l4" if self.compute_target == "cloud_run" else
"control_plane"
342     +     +         )

```

```

343     +         if seg != "cloud_run_l4":
344     +             raise ValueError(
345     +                 "deployment.phase_placement.validation='cloud_run_l4' requires
segment to also "
346     +                 "run on the L4 ? the worker can only validate a clip it also
detects. Set "
347     +                 "phase_placement.segment='cloud_run_l4' (or
compute_target='cloud_run'), or keep "
348     +                 "validation on the control_plane."
349     +             )
350     +         return self
351
352
353     class BillingConfig(BaseModel):
354     diff --git a/backend/config/placement.py b/backend/config/placement.py
355     new file mode 100644
356     index 0000000..cda3ca5
357     --- /dev/null
358     +++ b/backend/config/placement.py
359     @@ -0,0 +1,59 @@
360     +"""Phase ? machine placement resolution (#521).
361     +
362     +A *phase placement* answers one question per pipeline phase ? **where does the heavy
compute for
363     +this phase run?** ? with a config knob (`deployment.phase_placement`) instead of
code. The three
364     +placeable phases and their values:
365     +
366     +     validation ? control_plane | cloud_run_l4
367     +     segment    ? control_plane | cloud_run_l4    (a.k.a. "detect")
368     +     compile     ? control_plane | cloud_run_l4    (a.k.a. "stitch")
369     +
370     +`control_plane` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
`cloud_run_l4`
371     +dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already
uses). This module
372     +is the SINGLE source of truth so `backend.orchestration` (segment + compile) and the
worker never
373     +re-derive the mapping.
374     +
375     +**Default = today's behaviour (pure / additive).** When a phase's knob is unset
(`None`) the
376     +resolver reproduces the pre-#521 placement:
377     +
378     +     validation ? control_plane                                (the authoritative
gate, #512/#513)
379     +     segment    ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
380     +     compile     ? control_plane                                (the in-process
ffmpeg stitch)
381     +
382     +so a config with no `phase_placement` block behaves byte-identically to before this
module existed.
383     +"""
384     +
385     +from __future__ import annotations
386     +
387     +from typing import Any
388     +
389     +# Phase identities (kept as constants so callers don't stringly-type the phase names).
390     +PHASE_VALIDATION = "validation"
391     +PHASE_SEGMENT = "segment"
392     +PHASE_COMPILE = "compile"
393     +PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
394     +

```

```

395     ## Placement targets.
396     +PLACE_CONTROL_PLANE = "control_plane"
397     +PLACE_CLOUD_L4 = "cloud_run_l4"
398     +PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
399     +
400     +
401     +def resolve_phase_placement(cfg: Any, phase: str) -> str:
402     +     """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
403     +
404     +     Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the
legacy default
405     +     (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an
absent
406     +     ``deployment`` / ``phase_placement`` degrades to the legacy default.
407     +     """
408     +     deployment = getattr(cfg, "deployment", None)
409     +     placement = getattr(deployment, "phase_placement", None)
410     +     explicit = getattr(placement, phase, None) if placement is not None else None
411     +     if explicit:
412     +         return str(explicit)
413     +     # Legacy defaults ? reproduce today's placement exactly.
414     +     if phase == PHASE_SEGMENT:
415     +         compute_target = (getattr(deployment, "compute_target", "") or "").strip()
416     +         return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
417     +     # validation + compile default to the control-plane (their pre-#521 home).
418     +     return PLACE_CONTROL_PLANE
419 diff --git a/backend/config/presets.py b/backend/config/presets.py
420 index 77b20e8..0ea0c88 100644
421 --- a/backend/config/presets.py
422 +++ b/backend/config/presets.py
423 @@ -49,7 +49,15 @@ PROFILE_PRESETS: dict[str, dict[str, Any]] = {
424     "storage": {"backend": "local"},
425     },
426     "cloud-serving": {
427     -     "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
428     +     "deployment": {
429     +         "profile": "cloud-serving",
430     +         "compute_target": "cloud_run",
431     +         # #521: run the compile/stitch on the L4 worker (NVENC + 8 vCPU), not the
CPU-only
432     +         # 2-vCPU control-plane where the 2026-07-08 CUJ measured ~13 min for an
11-clip 4K reel.
433     +         # (validation stays on the control-plane ? the authoritative #512/#513
gate; segment is
434     +         # already cloud_run_l4 via compute_target.)
435     +         "phase_placement": {"compile": "cloud_run_l4"},
436     +     },
437     +     # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172
floor (D11).
438     +     # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the
paid L4
439     +     # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
diff --git a/backend/orchestration.py b/backend/orchestration.py
440 index 8b7851e..46ffaa4 100644
441 --- a/backend/orchestration.py
442 +++ b/backend/orchestration.py
443 +++ b/backend/orchestration.py
444 @@ -26,11 +26,20 @@ import os
445     import re
446     import tempfile
447     import threading
448     +import uuid
449     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
450     from urllib.parse import quote
451
452     from backend import storage

```

```

453 -from backend.config import AppConfig, StitchingConfig
454 +from backend.config import (
455 +     PHASE_COMPILE,
456 +     PHASE_SEGMENT,
457 +     PHASE_VALIDATION,
458 +     PLACE_CLOUD_L4,
459 +     AppConfig,
460 +     StitchingConfig,
461 +     resolve_phase_placement,
462 +)
463     from backend.pipeline.segmenters.base import segmenter_registry
464     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
465     from backend.pipeline.validators.base import ValidationResult, registry
466 @@ -206,6 +215,24 @@ def _cloud_available(cfg) -> bool:
467     return bool(has_job and out_root)
468
469
470 +def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
471 +    """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
472 +    ``maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
473 +    ``execute_processing`` can
474 +    decide whether to DELEGATE validation to the worker
475 +    (``phase_placement.validation``). A per-request
476 +    picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
477 +    cloud iff
478 +    ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
479 +    pre-dispatch reject
480 +    (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
481 +    remotely."""
482 +    choice = (req_compute or "").strip().lower()
483 +    if choice == "local":
484 +        return False
485 +    if choice == "cloud":
486 +        return _cloud_available(cfg)
487 +    return (
488 +        resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
489 +        and _cloud_available(cfg)
490 +    )
491
492 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
493     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
494     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
495     comes
496 @@ -505,11 +532,19 @@ def _maybe_dispatch_remote(
497     )
498     compute = get_compute_backend(cfg, target_override="cloud_run")
499     else:
500         compute = get_compute_backend(
501             cfg
502         ) # server default (default-OFF unless cloud_run)
503         # No explicit picker ? consult phase_placement.segment (#521). Its legacy
504         default is
505         # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
506         behaviour; setting
507         + # phase_placement.segment relocates DETECT by config alone (e.g. force it back
508         on the
509         + # control-plane, or onto the L4 on a default-local box wired for cloud).
510         + if (
511         +     resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
512         +     and _cloud_available(cfg)
513         + ):
514         +     compute = get_compute_backend(cfg, target_override="cloud_run")
515         + else:
516         +     compute = None

```

```

509         if compute is None:
510             -         return None # default-OFF ? run the in-process segmenter below (unchanged)
511             +         return None # default-OFF / placed on control_plane ? the in-process segmenter
below
512
513             storage_cfg = getattr(cfg, "storage", None)
514
515             @@ -623,12 +658,20 @@ def _maybe_dispatch_remote(
516                 # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
517                 # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
518                 # then worker-rejected at its default 20.0 after a successful GPU dispatch).
519             +         # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
520             +         # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
521             +         # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
522             +         # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators
? see
523             +         # _execute_processing), so the worker must NOT skip: it becomes the gate.
524             +         worker_skips_validation = (
525             +             resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
526             +         )
527             spec = ComputeJobSpec(
528                 video_uri=req.video_path,
529                 out_uri=out_root,
530                 segmenter=seg_name,
531                 config_overrides=tuple(overrides),
532             -         skip_validation=True,
533             +         skip_validation=worker_skips_validation,
534             )
535             try:
536                 # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
537             @@ -811,12 +854,33 @@ def _execute_processing(
538                 # ``force=true`` bypasses the reuse and re-writes a fresh verdict below
(mirrors how it bypasses
539                 # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
540                 # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
541             -         val_fingerprint = registry.config_fingerprint(config)
542             +         #
543             +         # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
544             +         # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
545             +         # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
546             +         # 2-VCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
547             +         # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed
job. When detection
548             +         # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
549             +         # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
550             +         validate_on_control_plane = (
551             +             resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
552             +             or not _will_detect_on_cloud(config, getattr(req, "compute", None))
553             +         )
554             +         if not validate_on_control_plane:
555             +             logger.info(
556             +                 "[API] validation delegated to the L4 worker "

```

```

557 +                "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
558 +                "validator suite; the worker is the gate for %s.",
559 +                req.video_path,
560 +            )
561 +            val_fingerprint = (
562 +                registry.config_fingerprint(config) if validate_on_control_plane else ""
563 +            )
564 +            # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
565 +            # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
566 +            # failing the job. The cache is a pure speedup ? never a new failure mode.
567 +            reused_validation = False
568 -            if not req.force:
569 +            if validate_on_control_plane and not req.force:
570 +                try:
571 +                    cached = db.get_validation_cache(video_id)
572 +                    if cached is not None and cached.get("fingerprint") == val_fingerprint:
573 @@ -841,11 +905,11 @@ def _execute_processing(
574 +                    video_id,
575 +                    exc_info=True,
576 +                )
577 -            if not reused_validation:
578 +            if validate_on_control_plane and not reused_validation:
579 +                logger.info(f"Running validations for {req.video_path}...")
580 +                val_results, val_context = registry.run_all_with_context(local_video,
config)
581 +                failures = [r for r in val_results if not r.passed]
582 -            if not reused_validation:
583 +            if validate_on_control_plane and not reused_validation:
584 +                # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
585 +                # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
586 +                db.set_validation_cache(
587 @@ -1114,28 +1178,34 @@ def signed_reel_url(cfg, video_id: str, name: str) ->
Optional[str]:
588 +                return None
589
590
591 -def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
592 -    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
593 -    and return the {status, filepath, download_url} result the SPA polls for. Returns a
594 -    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
595 -    so the SPA always gets a clean verdict instead of a worker-crash error."""
596 -    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
597 +def _compile_wait_message(elapsed_sec: float) -> str:
598 +    """The live job.progress line while an L4 compile/stitch is in flight (#521;
mirrors
599 +    ``_wait_progress_message``). Minute-granular; under a minute reads as cold-
start."""
600 +    minutes = int(elapsed_sec) // 60
601 +    if minutes < 1:
602 +        return "stitching on cloud GPU (NVENC) ? starting up"
603 +    return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
604
605 -    notify = progress or (lambda stage: None)
606 -    params = json.loads(job.get("params") or "{}")
607 -    video_id = job.get("video_id") or ""
608 -    segment_ids = params.get("segment_ids") or []
609 -    locale = params.get("locale")

```

```

610
611     - notify("resolving")
612     - try:
613         - video, kept, out_name = _resolve_compile(
614             - db, cfg, video_id, segment_ids, params.get("output_filename", "")
615             - )
616         - except _CompileError as e:
617             - return {"status": "failed", "message": e.detail, "video_id": video_id}
618 +def _stitch_reel(
619     + cfg,
620     + stitching,
621     + video_id: str,
622     + source_ref: str,
623     + time_pairs: List[Tuple[float, float]],
624     + out_name: str,
625     + locale: Optional[str],
626     + notify,
627     +) -> Dict[str, Any]:
628     + """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
629     + (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
630     + needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
631     + with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the
source, runs the
632     + trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
633     + returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
634     + ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
635     + import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
636
637     - stitching = getattr(cfg, "stitching", None) or StitchingConfig()
638     - output_dir = (
639         - getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
640         - )
641     @@ -1147,8 +1217,8 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
642         - begin_padding=stitching.begin_padding,
643         - end_padding=stitching.end_padding,
644         - watermark=localized_watermark,
645     + encoder=getattr(stitching, "encoder", "libx264"),
646     + )
647     - time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
648
649     - notify("stitching")
650     - local_src = ""
651     @@ -1157,7 +1227,7 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
652     - # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
653     - # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
654     - local_src = storage.acquire_local_input(
655     -     - video["filepath"], getattr(cfg, "storage", None)
656     + source_ref, getattr(cfg, "storage", None)
657     + )
658     - # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
659     - # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
660     @@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any],

```



```

progress=None) -> Dict[str, A
661         finally:
662             if local_src:
663                 storage.cleanup_acquired_local_input(
664                     video["filepath"], local_src, getattr(cfg, "storage", None)
665                     + source_ref, local_src, getattr(cfg, "storage", None)
666                 )
667
668         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
669         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
670         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
671         - # still falls back to the local file on the same instance).
672         + # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
673         + # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
674         reel_uri = reel_object_uri(cfg, video_id, out_name)
675         if reel_uri:
676             try:
677                 @@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
678                 "filepath": out_path,
679                 "download_url": download_url,
680                 "video_id": video_id,
681                 + "reel_uri": reel_uri or "",
682             }
683         +
684         +
685         +def _dispatch_compile_remote(
686         +     cfg,
687         +     db,
688         +     video_id: str,
689         +     segment_ids: List[int],
690         +     output_filename: str,
691         +     locale: Optional[str],
692         +     notify,
693         + ) -> Dict[str, Any]:
694         +     """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
695         +     control-plane owns the DB, so it RESOLVES the reel's segments here
(`_resolve_compile`), stages a
696         +     small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
697         +     to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
698         +     done-marker, and returns the same ``{status, filepath, download_url}`` shape the
SPA expects. The
699         +     worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
700         +     ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
701         +     from backend.compute import CompileJobSpec, get_compute_backend
702         +
703         +     notify("resolving")
704         +     try:
705         +         video, kept, out_name = _resolve_compile(
706         +             db, cfg, video_id, segment_ids, output_filename
707         +         )
708         +     except _CompileError as e:
709         +         return {"status": "failed", "message": e.detail, "video_id": video_id}
710         +
711         +     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
712         +     storage_cfg = getattr(cfg, "storage", None)
713         +     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()

```

```

714 +     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
715 +     if compute is None:
716 +         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
717 +         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail
? a slow reel
718 +         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
719 +         logger.warning(
720 +             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
721 +             "stitching in-process on the control-plane instead.",
722 +             out_root,
723 +         )
724 +         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
725 +         return _stitch_reel(
726 +             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
727 +         )
728 +
729 +     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-
safe on the wire);
730 +     # the stitching config is the control-plane's RESOLVED one, with the encoder
swapped to the L4's
731 +     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-
encodes the reel.
732 +     stitch_dump = stitching.model_dump()
733 +     stitch_dump["encoder"] = getattr(
734 +         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
735 +     )
736 +     spec_payload = {
737 +         "video_id": video_id,
738 +         "video_uri": video["filepath"],
739 +         "output_name": out_name,
740 +         "locale": locale,
741 +         "time_pairs": [
742 +             [float(s["start_time"]), float(s["end_time"])] for s in kept
743 +         ],
744 +         "stitching": stitch_dump,
745 +     }
746 +     token = uuid.uuid4().hex
747 +     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
748 +     try:
749 +         notify("staging compile spec")
750 +         with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
751 +             local_spec = os.path.join(td, "spec.json")
752 +             with open(local_spec, "w", encoding="utf-8") as f:
753 +                 json.dump(spec_payload, f)
754 +             storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
755 +     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
756 +         logger.error(
757 +             "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
758 +         )
759 +     return {
760 +         "status": "failed",
761 +         "message": f"cloud compile: could not stage the compile spec: {e}",
762 +         "video_id": video_id,
763 +     }
764 +
765 +     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
766 +     # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB

```

knobs for detect.

```
767 +     overrides = [f"storage.output_root={out_root}"]
768 +     backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
769 +     if backend_name:
770 +         overrides.append(f"storage.backend={backend_name}")
771 +     spec = CompileJobSpec(
772 +         out_uri=out_root,
773 +         spec_uri=spec_uri,
774 +         video_id=video_id,
775 +         output_name=out_name,
776 +         config_overrides=tuple(overrides),
777 +     )
778 +
779 +     notify("dispatching stitch (cloud_run)")
780 +     logger.info(
781 +         "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
782 +         video_id,
783 +         out_root,
784 +         len(kept),
785 +     )
786 +     try:
787 +         result = compute.run_compile(
788 +             spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
789 +         )
790 +     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
791 +         logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
792 +         return {
793 +             "status": "failed",
794 +             "message": f"cloud compile dispatch error: {e}",
795 +             "video_id": video_id,
796 +         }
797 +     if result.returncode != 0:
798 +         msg = str(
799 +             result.report.get("message")
800 +             or f"cloud compile job failed (returncode {result.returncode})."
801 +         )
802 +         return {"status": "failed", "message": msg, "video_id": video_id}
803 +
804 +     download_url = str(
805 +         result.report.get("download_url")
806 +         or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
807 +     )
808 +     logger.info(
809 +         "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
810 +         result.execution,
811 +         result.outputs_uri,
812 +         video_id,
813 +     )
814 +     notify("finished")
815 +     return {
816 +         "status": "success",
817 +         "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
818 +         "download_url": download_url,
819 +         "video_id": video_id,
820 +         "compute": {
821 +             "path": "cloud_run",
822 +             "execution": result.execution,
823 +             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
824 +             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
825 +         },
826 +     }
827 +
828 +
```

```

829     +def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
830     +     """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
831     +     the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
832     +     result for an EXPECTED problem (missing video / no segments / a stitch error) so
the SPA always gets
833     +     a clean verdict instead of a worker-crash error.
834     +
835     +     #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
836     +     dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
837     +     (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
838     +     2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
839     +     notify = progress or (lambda stage: None)
840     +     params = json.loads(job.get("params") or "{}")
841     +     video_id = job.get("video_id") or ""
842     +     segment_ids = params.get("segment_ids") or []
843     +     locale = params.get("locale")
844     +     output_filename = params.get("output_filename", "")
845     +
846     +     if (
847     +         resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
848     +         and _cloud_available(cfg)
849     +     ):
850     +         return _dispatch_compile_remote(
851     +             cfg, db, video_id, segment_ids, output_filename, locale, notify
852     +         )
853     +
854     +     notify("resolving")
855     +     try:
856     +         video, kept, out_name = _resolve_compile(
857     +             db, cfg, video_id, segment_ids, output_filename
858     +         )
859     +     except _CompileError as e:
860     +         return {"status": "failed", "message": e.detail, "video_id": video_id}
861     +
862     +     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
863     +     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
864     +     return _stitch_reel(
865     +         cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
866     +     )
867     diff --git a/backend/pipeline/stitcher/compiler.py
b/backend/pipeline/stitcher/compiler.py
868     index 65d1c57..5c7229e 100644
869     --- a/backend/pipeline/stitcher/compiler.py
870     +++ b/backend/pipeline/stitcher/compiler.py
871     @@ -23,6 +23,49 @@ _BUNDLED_FONT = (
872     +     Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
873     + )
874
875     +#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
876     +#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
877     +_ENCODER_AVAILABLE: Dict[str, bool] = {}
878     +
879     +
880     +def _probe_encoder_available(encoder: str) -> bool:
881     +     """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
882     +     lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA

```

```

GPU, so a
883 +     real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
884 +     runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
885 +     if encoder in _ENCODER_AVAILABLE:
886 +         return _ENCODER_AVAILABLE[encoder]
887 +     ok = False
888 +     # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the
configured ffmpeg
889 +     # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
890 +     configured = ffmpeg_timeout_sec()
891 +     probe_timeout = min(30.0, float(configured)) if configured else 30.0
892 +     try:
893 +         proc = subprocess.run(
894 +             [
895 +                 ffmpeg_bin(),
896 +                 "-hide_banner",
897 +                 "-f",
898 +                 "lavfi",
899 +                 "-i",
900 +                 "color=c=black:s=64x64:d=0.1",
901 +                 "-frames:v",
902 +                 "1",
903 +                 "-c:v",
904 +                 encoder,
905 +                 "-f",
906 +                 "null",
907 +                 "-",
908 +             ],
909 +             capture_output=True,
910 +             timeout=probe_timeout,
911 +         )
912 +         ok = proc.returncode == 0
913 +     except Exception: # noqa: BLE001 ? a probe failure/timeout just means "not usable
? fall back"
914 +         ok = False
915 +         _ENCODER_AVAILABLE[encoder] = ok
916 +         return ok
917 +
918
919     class VideoCompiler:
920         def __init__(
921             @@ -31,6 +74,7 @@ class VideoCompiler:
922             end_padding: float = 1.0,
923             begin_padding: float = 0.5,
924             watermark: Optional[Any] = None,
925             + encoder: str = "libx264",
926             ):
927             self.output_dir = output_dir
928             self.end_padding = end_padding
929             @@ -38,6 +82,11 @@ class VideoCompiler:
930             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
931             # normalized to a dict (or None) so this module stays config-package-agnostic.
932             self.watermark = self._normalize_watermark(watermark)
933             + # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
934             + # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
935             + # re-encode the watermark forces. The requested encoder is PROBED once per run
936             + # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run
it.
937             + self.encoder = (encoder or "libx264").strip() or "libx264"
938             os.makedirs(output_dir, exist_ok=True)
939

```

```

940         @staticmethod
941         @@ -225,6 +274,30 @@ class VideoCompiler:
942             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
943             return drawtext
944
945     + def _select_encoder(self) -> Tuple[List[str], str]:
946     +     """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
947     +     chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec
and the final
948     +     concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-
encode; if the
949     +     requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
950     +     libx264 so a misconfig never nukes the reel."""
951     +     enc = self.encoder
952     +     if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
953     +         logger.warning(
954     +             "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
955     +             "libx264 (CPU). On the L4 worker this should be available.",
956     +             enc,
957     +         )
958     +     enc = "libx264"
959     +     if enc == "libx264":
960     +         # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
961     +         return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
962     +         # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
963     +         # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
964     +         return (
965     +             ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
966     +             enc,
967     +         )
968     +
969     def _trim_one_segment(
970         self,
971         idx: int,
972         @@ -235,6 +308,7 @@ class VideoCompiler:
973         tmp_dir: str,
974         raw_video_path: str,
975         watermark_filter: Optional[str],
976     +     video_codec_args: Optional[List[str]] = None,
977     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
978     +     """Trim a single segment into its own clip file (extracted verbatim from the
per-segment body of compile_highlights). Returns (clip_path, skip_record): on
979     +
980     @@ -297,6 +371,13 @@ class VideoCompiler:
981         f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
982     +     )
983
984     +     # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
985     +     # directly without resolving an encoder ? keeps this method's old behaviour
intact.
986     +     codec_args = (
987     +         video_codec_args
988     +         if video_codec_args is not None
989     +         else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
990     +     )
991     +     base_cmd = [
992     +         ffmpeg_bin(),
993     +         "-y",
994     @@ -306,12 +387,7 @@ class VideoCompiler:

```

```

995         str(round(padded_end, 3)),
996         "-i",
997         raw_video_path,
998         "-c:v",
999         "libx264",
1000         "-preset",
1001         "superfast",
1002         "-crf",
1003         "22",
1004         + *codec_args,
1005         "-c:a",
1006         "aac",
1007         "-b:a",
1008     @@ -445,6 +521,11 @@ class VideoCompiler:
1009         watermark_filter = self._watermark_filter(tmp_dir)
1010         if watermark_filter:
1011             logger.info("[watermark] applying branding overlay to each clip.")
1012         + # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
1013         + # and the final concat stays -c copy (#521).
1014         + video_codec_args, chosen_encoder = self._select_encoder()
1015         + if chosen_encoder != "libx264":
1016         +     logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
1017         for idx, (raw_start, raw_end) in enumerate(segments):
1018             clip_path, skip_record = self._trim_one_segment(
1019                 idx,
1020             @@ -455,6 +536,7 @@ class VideoCompiler:
1021                 tmp_dir,
1022                 raw_video_path,
1023                 watermark_filter,
1024             + video_codec_args,
1025             )
1026             if clip_path is not None:
1027                 temp_clips.append(clip_path)
1028 diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
1029 index 6d3420b..79db808 100644
1030 --- a/backend/worker/__main__.py
1031 +++ b/backend/worker/__main__.py
1032 @@ -419,6 +419,108 @@ def cmd_infer(args: argparse.Namespace) -> int:
1033     return 0
1034
1035
1036 +def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
1037 +    """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
1038 +    resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
1039 +    A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
1040 +    local = storage.fetch(
1041 +        spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
1042 +    )
1043 +    with open(local, encoding="utf-8") as f:
1044 +        data = json.load(f)
1045 +        if not isinstance(data, dict):
1046 +            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
1047 +        return data
1048 +
1049 +
1050 +def _write_compile_marker(
1051 +    done_uri: str,
1052 +    result: dict,
1053 +    video_id: str,
1054 +    returncode: int,

```

10. assistant (2026-07-08T06:04:38.581Z)

Let me read the rest of the diff, especially the worker and the tests.

11. user (2026-07-08T06:04:40.990Z)

```
1055 +     storage_cfg: dict,
1056 +     scratch: str,
1057 +) -> None:
1058 +     """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
1059 +     dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
1060 +     try:
1061 +         marker = {
1062 +             "video_id": video_id,
1063 +             "returncode": returncode,
1064 +             "status": result.get("status", "failed"),
1065 +             "download_url": result.get("download_url", ""),
1066 +             "reel_uri": result.get("reel_uri", ""),
1067 +             "message": result.get("message", ""),
1068 +         }
1069 +         local = os.path.join(scratch, "_compile_done.json")
1070 +         with open(local, "w", encoding="utf-8") as f:
1071 +             json.dump(marker, f)
1072 +         storage.put(local, done_uri, config=storage_cfg)
1073 +         logger.info("[worker] wrote compile marker -> %s", done_uri)
1074 +     except Exception as e: # never fail a good stitch because the marker push hiccuped
1075 +         logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
1076 +
1077 +
1078 +def cmd_compile(args: argparse.Namespace) -> int:
1079 +     """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU).
The stitch is the
1080 +     2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
1081 +     the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
1082 +     the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses
(so there is no
1083 +     forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
1084 +     bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
1085 +     from backend import orchestration
1086 +     from backend.config import StitchingConfig
1087 +
1088 +     config = load_config(args.config) if args.config else load_config()
1089 +     _apply_overrides(config, args.set)
1090 +     storage_cfg = config.storage.model_dump()
1091 +
1092 +     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
1093 +     os.makedirs(scratch, exist_ok=True)
1094 +     # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
1095 +     config.output.output_dir = scratch
1096 +     # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
1097 +     # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
1098 +     # destination even if the override were dropped.
1099 +     if not (config.storage.output_root or "").strip():
1100 +         config.storage.output_root = args.out_uri
1101 +
```



```

1102 + spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
1103 + video_id = str(spec.get("video_id") or "")
1104 + out_name = str(spec.get("output_name") or "")
1105 + source_ref = str(spec.get("video_uri") or "")
1106 + locale = spec.get("locale")
1107 + time_pairs = [
1108 +     (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
1109 + ]
1110 + stitching = StitchingConfig(**(spec.get("stitching") or {}))
1111 +
1112 + print(
1113 +     f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
1114 +     f"encoder={stitching.encoder} out={out_name}"
1115 + )
1116 +
1117 + result = orchestration._stitch_reel(
1118 +     config,
1119 +     stitching,
1120 +     video_id,
1121 +     source_ref,
1122 +     time_pairs,
1123 +     out_name,
1124 +     locale,
1125 +     lambda stage: print(f"[worker] compile: {stage}"),
1126 + )
1127 + status = str(result.get("status") or "failed")
1128 + rc = 0 if status == "success" else 3
1129 + if status != "success":
1130 +     print(f"[worker] compile FAILED: {result.get('message')}")
1131 +
1132 + # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
1133 + if getattr(args, "done_uri", None):
1134 +     _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg,
scratch)
1135 + return rc
1136 +
1137 +
1138 #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
1139 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
1140 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
1141 @@ -760,6 +862,43 @@ def build_parser() -> argparse.ArgumentParser:
1142 )
1143 pi.set_defaults(func=cmd_infer)
1144
1145 + pc = sub.add_parser(
1146 +     "compile",
1147 +     help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
1148 + )
1149 + pc.add_argument(
1150 +     "--spec-uri",
1151 +     required=True,
1152 +     help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
1153 +     "config + source uri + video_id)",
1154 + )
1155 + pc.add_argument(
1156 +     "--out-uri",
1157 +     default="",
1158 +     help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
1159 +     "Also forwarded via --set storage.output_root; either resolves the reel path.",

```

```

1160 + )
1161 + pc.add_argument(
1162 +     "--done-uri",
1163 +     default=None,
1164 +     help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
1165 +     "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
1166 + )
1167 + pc.add_argument(
1168 +     "--config", default=None, help="config.json path (default: ./config.json)"
1169 + )
1170 + pc.add_argument(
1171 +     "--scratch", default=None, help="local scratch dir (default: a temp dir)"
1172 + )
1173 + pc.add_argument(
1174 +     "--set",
1175 +     action="append",
1176 +     default=[],
1177 +     metavar="k=v",
1178 +     help="config override, e.g. --set storage.output_root=gs://bucket",
1179 + )
1180 + pc.set_defaults(func=cmd_compile)
1181 +
1182     pt = sub.add_parser(
1183         "train",
1184         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
1185         diff --git a/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
b/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
1186         new file mode 100644
1187         index 00000000..d0f7922
1188         --- /dev/null
1189         +++ b/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
1190         @@ -0,0 +1,169 @@
1191         +# Phase ? machine placement + compile-on-L4 (#521)
1192         +
1193         +> **Status:** ? **SHIPPED (code)** . **Owner:** Avi . **Created:** 2026-07-08 .
**Issue:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
1194         +> **Relation:** extends [COMPUTE_DECOUPLED_SERVING](./README.md) (the `ComputeBackend`
dispatch seam) and **refines D16(b)** (NVENC-encode-deferred-behind-a-config-knob) now that the
2026-07-08 live CUJ supplies the cost data D16(b) said to revisit on.
1195         +> **Evidence:** `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`
$9 (2026-07-08 live CUJ, rev `rally-control-plane-00019`).
1196         +
1197         +---
1198         +
1199         +## 0. TL;DR
1200         +
1201         +The compile/stitch phase used to run **in-process on the CPU-only 2-vCPU control-
plane**, where the
1202         +2026-07-08 live CUJ measured **~12m55s** for an 11-clip / ~80 s reel from a 1.83 GiB 4K
source
1203         +(~45?90 s per ~7 s clip ? the per-clip branding watermark forces a full 4K re-encode
with **no
1204         +NVENC**). Validation (183 s) runs there too; both serialize on `_LOCAL_COMPUTE_LANE`,
so one run pins
1205         +the control-plane for ~16 min ? the API can't scale past a handful of concurrent users.
1206         +
1207         +#521 makes **where each pipeline phase runs a config edit, not a code change**, and
moves the stitch
1208         +to the **L4 worker (NVENC + 8 vCPU)** the way detection already dispatches:
1209         +
1210         +1. **`deployment.phase_placement`** ? a per-phase knob (`validation` / `segment` /
`compile`) with
1211         + values `control_plane | cloud_run_l4`. Unset ? **today's placement, byte-

```

```

identical**.
1212 +2. **compile ? L4** ? a `kind='compile'` Cloud Run dispatch mirroring the detect path:
the
1213 + control-plane resolves the reel's segments, stages a compile-spec to the bucket,
dispatches
1214 + `python -m backend.worker compile` on the L4, bucket-polls a done-marker, and
returns the same
1215 + `{status, filepath, download_url}` shape the SPA already expects.
1216 +3. **NVENC on the L4** ? `deployment.cloud_stitch_encoder` (default `h264_nvenc`) is
forwarded onto
1217 + the worker's `stitching.encoder`; libx264 stays the everywhere-safe fallback (auto-
probed).
1218 +
1219 +The cloud-serving preset opts compile onto the L4
(`phase_placement.compile=cloud_run_l4`).
1220 +
1221 +---
1222 +
1223 +## 1. The placement model
1224 +
1225 +`resolve_phase_placement(cfg, phase)` (in `backend/config/placement.py`) is the
**single source of
1226 +truth** ? orchestration and the worker never re-derive the mapping. Precedence: an
explicit
1227 +`deployment.phase_placement.<phase>` wins; otherwise the **legacy default** reproduces
pre-#521
1228 +behaviour exactly:
1229 +
1230 +| phase | knob unset (legacy default) | meaning |
1231 +|---|---|---|
1232 +| `validation` | `control_plane` | the authoritative gate (#512/#513) stays on the
control-plane |
1233 +| `segment` (detect) | `cloud_run_l4` **iff** `compute_target==cloud_run`, else
`control_plane` | today's DETECT dispatch |
1234 +| `compile` (stitch) | `control_plane` | the in-process ffmpeg stitch |
1235 +
1236 +So a config with **no** `phase_placement` block behaves exactly as before this existed.
To relocate a
1237 +phase, set its knob:
1238 +
1239 +```jsonc
1240 +// Move the stitch to the L4 (what the cloud-serving preset does):
1241 +{ "deployment": { "compute_target": "cloud_run",
1242 +                  "phase_placement": { "compile": "cloud_run_l4" } } }
1243 +
1244 +// Force detection back onto the control-plane on a cloud box (e.g. a debugging run):
1245 +{ "deployment": { "compute_target": "cloud_run",
1246 +                  "phase_placement": { "segment": "control_plane" } } }
1247 +
1248 +// Move validation off the control-plane too (must pair with a cloud segment ? see §4):
1249 +{ "deployment": { "compute_target": "cloud_run",
1250 +                  "phase_placement": { "validation": "cloud_run_l4" } } }
1251 +```
1252 +
1253 +`control_plane` runs the phase in-process on the (CPU-only) control-plane service;
`cloud_run_l4`
1254 +dispatches it to the GPU/NVENC Cloud Run worker Job.
1255 +
1256 +---
1257 +
1258 +## 2. compile ? L4 dispatch (the flow)
1259 +
1260 +`orchestration._execute_compile` consults `resolve_phase_placement(cfg, "compile")`.
When it resolves
1261 +to `cloud_run_l4` **and** cloud dispatch is available (`_cloud_available`), it calls

```

```

1262 +`_dispatch_compile_remote`; otherwise it stitches in-process via the shared
`_stitch_reel` core
1263 +(unchanged).
1264 +
1265 +`_dispatch_compile_remote` (mirrors `_maybe_dispatch_remote`):
1266 +
1267 +1. **Resolve** the reel's segments HERE ? the control-plane owns the DB
(`_resolve_compile` ?
1268 + `(video, kept, out_name)`), applying the `stitching.min_shot_count` floor as before.
1269 +2. **Stage a compile-spec** to `/_compile/<token>/spec.json` ? the
resolved
1270 + `time_pairs`, the control-plane's resolved `stitching` config (watermark + padding,
with `encoder`
1271 + swapped to `cloud_stitch_encoder`), the source `video_uri`, `video_id`,
`output_name`, `locale`.
1272 + The worker therefore needs **no control-plane DB** ? everything rides the spec.
1273 +3. **Dispatch** a `compile` Cloud Run Job execution (`CompileJobSpec` ?
`CloudRunCompute.run_compile`)
1274 + running `python -m backend.worker compile --spec-uri ? --done-uri ? --set
storage.output_root=?`,
1275 + forwarding the bucket coordinates (the worker Job has no `DEPLOYMENT_PROFILE`).
1276 +4. **Bucket-poll** the done-marker at `/_compile/<token>.done` ? the SAME
1277 + `_await_marker` machinery `run_infer` uses (crash-fast on a terminal-without-marker
execution,
1278 + rescue-or-cancel on a poll timeout). Distinct `_compile/` prefix so a compile and a
detection for
1279 + one bucket never collide on a marker.
1280 +5. The worker (`cmd_compile`) reads the spec, stitches the reel via the **shared
`_stitch_reel`
1281 + core** (no forked pipeline), mirror-uploads it to
`<output_root>/highlights/<video_id>/<name>`,
1282 + and writes the marker (rc + `download_url` + `reel_uri`).
1283 +6. The control-plane returns `{status, filepath, download_url, video_id}` ? the reel is
already in the
1284 + bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker
rc?0 ?
1285 + `status='failed'` with the worker's message (clean SPA verdict, never a crash).
1286 +
1287 +**Fallback:** if compile is placed on the L4 but the box isn't actually cloud-wired (no
output bucket
1288 +/ Cloud Run env), `_dispatch_compile_remote` logs and stitches in-process ? a slow reel
beats no reel.
1289 +
1290 +---
1291 +
1292 +## 3. NVENC on the L4 ? and the D5/D16(b) parity note
1293 +
1294 +The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding
watermark forces
1295 +(~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521
adds an encoder
1296 +knob:
1297 +
1298 +- `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) |
`h264_nvenc` |
1299 + `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a
1-frame lavfi test
1300 + encode) and **falls back to libx264** if this ffmpeg build can't run it, so a
misconfig / non-GPU
1301 + box never breaks. All clips share the one chosen encoder ? the concat stays `-c
copy`.
1302 +- `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch
forwards onto
1303 + the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4
stitch on CPU.

```

```

1304 +- The control-plane's own local stitch always uses `stitching.encoder` (libx264 by
1305 + CP has no GPU.
1306 +
1307 Parity note (honest): [README.md](./README.md) D5/D16(b) kept stitch encode =
1308 libx264 to
1309 +protect a byte-exact stitch parity guarantee, and explicitly deferred NVENC encode
1310 behind a config
1311 +knob, "revisit only if M7 cost data justifies relaxing parity." The 2026-07-08 CUJ
1312 (~13 min stitch,
1313 +a hard scaling wall past ~5 concurrent users) is that cost data. #521 lands exactly
1314 the deferred
1315 +shape ? NVENC behind a config knob, libx264 the default, config-reversible per
1316 phase. Enabling
1317 +NVENC relaxes byte-identity at the reel ENCODE layer only (a cosmetic, shareable
1318 artifact); the
1319 WINDOW/rally-level parity (the analytical output) is untouched ? detection and
1320 windowing are
1321 +unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.
1322 +
1323 +> Follow-up (tracked, not in #521's first cut): a pre-rendered / overlay-filter
1324 watermark so the
1325 +> stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).
1326 +
1327 +---
1328 +
1329 ## 4. Validation placement + the "nobody validates" guard
1330 +
1331 +`phase_placement.validation` moves the CPU-bound validator suite off the 2-vCPU
1332 control-plane too,
1333 +via the existing `--skip-validation` lever (#513):
1334 +
1335 +- control_plane (default): the control-plane runs the validators and is the
1336 authoritative gate;
1337 + the worker skips its redundant re-validation (`skip_validation=True`) ? today's
1338 #512/#513
1339 + behaviour, byte-identical.
1340 +- cloud_run_l4: when detection actually dispatches to the L4, the control-
1341 plane delegates
1342 + ? it does NOT run the validator suite (`_execute_processing` skips it), and the
1343 dispatch carries
1344 + no `--skip-validation`, so the worker validates and rejects a bad clip via a
1345 non-zero rc
1346 + (surfaced as a failed job).
1347 +
1348 Footgun guard: a clip can only be validated on the worker if the worker also
1349 detects it there.
1350 +Two protections:
1351 +
1352 +1. Config-load guard (`DeploymentConfig` model-validator):
1353 `validation=cloud_run_l4` requires the
1354 + effective segment placement to also be `cloud_run_l4` (explicit, or via
1355 `compute_target=cloud_run`)
1356 + ? else config load fails with an actionable error.
1357 +2. Runtime guard (`_will_detect_on_cloud`): if a per-request `compute='local'`
1358 forces detection
1359 + in-process (no worker), the control-plane still validates ? never leave a clip
1360 unvalidated
1361 + because nobody ran the gate.
1362 +
1363 +---
1364 +
1365 ## 5. Files & tests
1366 +
1367 +| Area | Code | Tests |

```

```

1349  +|---|---|---|
1350  +| Placement resolver + config | `backend/config/placement.py`,
`backend/config/models.py` (`PhasePlacementConfig`, `DeploymentConfig.phase_placement` /
`cloud_stitch_encoder`, guard) | `tests/test_phase_placement.py`,
`tests/test_config_deployment.py` |
1351  +| Preset | `backend/config/presets.py` (`cloud-serving` ?
`phase_placement.compile=cloud_run_l4`) | `tests/test_phase_placement.py` |
1352  +| compile dispatch | `backend/orchestration.py` (`_execute_compile`,
`_dispatch_compile_remote`, `_stitch_reel`, `_will_detect_on_cloud`) |
`tests/test_compile_dispatch.py` |
1353  +| compute backend | `backend/compute/base.py` (`CompileJobSpec`, `run_compile`),
`backend/compute/cloud_run.py` (`run_compile`, `_await_marker`) |
`tests/test_compute_backend.py` |
1354  +| worker | `backend/worker/__main__.py` (`compile` subcommand, `cmd_compile`) |
`tests/test_worker.py` |
1355  +| NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`,
`_select_encoder`, `_probe_encoder_available`) | `tests/test_compiler.py` |
1356  +| validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip
flag, `_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
1357  +
1358  +All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the
staged spec,
1359  +writes the reel + done-marker into a local "bucket"), mirroring
`tests/test_api_cloud_dispatch.py`.
1360  diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md
b/docs/COMPUTE_DECOUPLED_SERVING/README.md
1361  index e60d644..e17cf71 100644
1362  --- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
1363  +++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
1364  @@ -6,6 +6,7 @@
1365  > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
1366  > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
1367  > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
1368  +> **#521 (2026-07-08):** phase?machine placement is now **config-driven** and
compile/stitch is **relocatable** to the L4 worker** (NVENC) ? see
[PHASE_PLACEMENT.md](../PHASE_PLACEMENT.md). This refines **D16(b)** (NVENC-encode deferred
behind a config knob) now that the live CUJ (§9 of the validation-latency runlog) supplies the
cost data D16(b) said to revisit on.
1369
1370  ---
1371
1372  diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
1373  index 5baaf94..d0b8604 100644
1374  --- a/tests/test_api_cloud_dispatch.py
1375  +++ b/tests/test_api_cloud_dispatch.py
1376  @@ -1103,3 +1103,73 @@ def test_wait_progress_message_is_honest_and_minute_granular():
1377      assert "wasb_hybrid" in early and "starting up" in early
1378      later = _wait_progress_message("wasb_hybrid", 125.0)
1379      assert later.endswith("2 min elapsed")
1380  +
1381  +
1382  +# --- #521: phase_placement.validation (delegate validation to the L4 worker)
-----
1383  +
1384  +
1385  +def test_validation_delegated_to_worker_when_placed_on_l4(cloud_env):
1386  +    """With phase_placement.validation=cloud_run_l4 (+ segment on the L4), the control-
plane does NOT

```

```

1387 +     run its validator suite ? it delegates to the worker, which validates (dispatch
carries NO
1388 +     --skip-validation). This is the #521 scaling move: get the CPU-bound validation off
the 2-VCPU CP. """
1389 +     db, bucket, monkeypatch = cloud_env
1390 +     ran = {"n": 0}
1391 +
1392 +     def _spy(*a, **k):
1393 +         ran["n"] += 1
1394 +         return _pass(), {}
1395 +
1396 +     monkeypatch.setattr(m.registry, "run_all_with_context", _spy)
1397 +     cfg = AppConfig.model_validate(
1398 +         {
1399 +             "deployment": {
1400 +                 "compute_target": "cloud_run",
1401 +                 "phase_placement": {"validation": "cloud_run_l4"},
1402 +             },
1403 +             "storage": {"backend": "local", "output_root": str(bucket)},
1404 +         }
1405 +     )
1406 +     fake = _FakeCloudClient()
1407 +     _inject_fake(monkeypatch, fake)
1408 +
1409 +     out = m._execute_processing(
1410 +         cfg, db, m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
1411 +     )
1412 +     assert out["status"] == "success"
1413 +     assert ran["n"] == 0 # the control-plane skipped its validators (delegated)
1414 +     assert "--skip-validation" not in fake.calls[0] # so the worker validates instead
1415 +
1416 +
1417 + def test_validation_not_delegated_when_detection_runs_in_process(cloud_env,
monkeypatch):
1418 +     """Footgun guard: validation is placed on the L4 but the request forces
compute='local' (detection
1419 +     runs in-process, no worker) ? the control-plane MUST still validate. Never leave a
clip unvalidated
1420 +     because nobody ran the gate. """
1421 +     db, bucket, _mp = cloud_env
1422 +     ran = {"n": 0}
1423 +
1424 +     def _spy(*a, **k):
1425 +         ran["n"] += 1
1426 +         return _pass(), {}
1427 +
1428 +     monkeypatch.setattr(m.registry, "run_all_with_context", _spy)
1429 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
1430 +     cfg = AppConfig.model_validate(
1431 +         {
1432 +             "deployment": {
1433 +                 "compute_target": "cloud_run",
1434 +                 "phase_placement": {"validation": "cloud_run_l4"},
1435 +             },
1436 +             "storage": {"backend": "local", "output_root": str(bucket)},
1437 +         }
1438 +     )
1439 +     fake = _FakeCloudClient()
1440 +     _inject_fake(monkeypatch, fake)
1441 +
1442 +     out = m._execute_processing(
1443 +         cfg,
1444 +         db,
1445 +         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",

```

```

compute="local"),
1446 + )
1447 + assert out["status"] == "success"
1448 + assert ran["n"] == 1 # in-process ? the control-plane validated (no worker to
delegate to)
1449 + assert fake.calls == [] # ran locally, never dispatched
1450 diff --git a/tests/test_compile_dispatch.py b/tests/test_compile_dispatch.py
1451 new file mode 100644
1452 index 0000000..6bdae1a
1453 --- /dev/null
1454 +++ b/tests/test_compile_dispatch.py
1455 @@ -0,0 +1,204 @@
1456 +"""Compile/stitch ? L4 worker dispatch (#521).
1457 +
1458 +Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker
by config
1459 +(``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ?
all GPU-free +
1460 +cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads
the staged
1461 +compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal
invariant: DEFAULT
1462 +(compile on the control-plane) still stitches in-process, byte-identically.
1463 +"""
1464 +
1465 +import json
1466 +import os
1467 +
1468 +import pytest
1469 +
1470 +import backend.compute as compute_pkg
1471 +import backend.orchestration as orch
1472 +from backend.compute.cloud_run import CloudRunJobClient
1473 +from backend.config import AppConfig
1474 +from backend.infrastructure.database import Database
1475 +from backend.infrastructure.execution_policy import ExecutionPolicy
1476 +
1477 +_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
1478 +
1479 +
1480 +class _FakeCompileClient(CloudRunJobClient):
1481 +    """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-
spec (proving the
1482 +    control-plane resolved + staged it), then write the reel done-marker into the local
bucket exactly
1483 +    where the real worker would, so the dispatcher's bucket-poll resolves."""
1484 +
1485 +    def __init__(self, *, returncode=0, video_id=None):
1486 +        self.calls: list[list[str]] = []
1487 +        self.spec: dict | None = None
1488 +        self.returncode = returncode
1489 +        self._video_id = video_id
1490 +
1491 +    def run_job(self, job_name, argv, *, region, project=None):
1492 +        self.calls.append(list(argv))
1493 +        spec_uri = argv[argv.index("--spec-uri") + 1]
1494 +        done_uri = argv[argv.index("--done-uri") + 1]
1495 +        with open(spec_uri, encoding="utf-8") as f:
1496 +            self.spec = json.load(f)
1497 +        vid = self._video_id or self.spec["video_id"]
1498 +        out_name = self.spec["output_name"]
1499 +        status = "success" if self.returncode == 0 else "failed"
1500 +        marker = {
1501 +            "video_id": vid,
1502 +            "returncode": self.returncode,

```



```

1503 +         "status": status,
1504 +         "download_url": f"/api/highlights/{vid}/{out_name}",
1505 +         "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
1506 +         "message": "" if self.returncode == 0 else "stitch blew up",
1507 +     }
1508 +     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
1509 +     with open(done_uri, "w", encoding="utf-8") as f:
1510 +         json.dump(marker, f)
1511 +     return "exec-compile-1"
1512 +
1513 +     def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
1514 +         raise AssertionError("cancel must never fire on the happy path")
1515 +
1516 +
1517 +def _inject_fake(monkeypatch, fake):
1518 +     real = compute_pkg.get_compute_backend
1519 +     monkeypatch.setattr(
1520 +         compute_pkg,
1521 +         "get_compute_backend",
1522 +         lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok",
**kw),
1523 +     )
1524 +
1525 +
1526 +@pytest.fixture
1527 +def compile_env(tmp_path, monkeypatch):
1528 +     """A control-plane DB with a video + 3 play segments, wired for cloud with compile
placed on the
1529 +     L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file
reads/writes)."""
1530 +     db = Database(str(tmp_path / "t.db"))
1531 +     bucket = tmp_path / "bucket"
1532 +     bucket.mkdir()
1533 +     video_id = "vidCompile"
1534 +     src = tmp_path / "source.mp4"
1535 +     src.write_bytes(b"SRC")
1536 +     db.add_video(video_id, str(src), "processed", [])
1537 +     sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
1538 +     sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
1539 +     sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
1540 +     cfg = AppConfig.model_validate(
1541 +         {
1542 +             "deployment": {
1543 +                 "compute_target": "cloud_run",
1544 +                 "phase_placement": {"compile": "cloud_run_l4"},
1545 +             },
1546 +             "storage": {"backend": "local", "output_root": str(bucket)},
1547 +         }
1548 +     )
1549 +     return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch
1550 +
1551 +
1552 +def _job(video_id, segment_ids, name="reel.mp4", locale=None):
1553 +     return {
1554 +         "id": "cj1",
1555 +         "video_id": video_id,
1556 +         "kind": "compile",
1557 +         "params": json.dumps(
1558 +             {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
1559 +         ),
1560 +     }
1561 +
1562 +
1563 +def test_compile_dispatches_to_l4_and_returns_reel(compile_env):

```

```

1564 + db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1565 + fake = _FakeCompileClient()
1566 + _inject_fake(monkeypatch, fake)
1567 +
1568 + out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))
1569 +
1570 + assert out["status"] == "success"
1571 + assert out["video_id"] == video_id
1572 + assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
1573 + assert out["compute"]["path"] == "cloud_run"
1574 + assert out["compute"]["execution"] == "exec-compile-1"
1575 + assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
1576 +
1577 +
1578 +def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
1579 +    """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
1580 +    start/end from the DB)
1581 +    and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so
1582 +    the worker
1583 +    hardware-encodes and needs no control-plane DB."""
1584 +    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1585 +    fake = _FakeCompileClient()
1586 +    _inject_fake(monkeypatch, fake)
1587 +
1588 +    orch._execute_compile(cfg, db, _job(video_id, seg_ids))
1589 +
1590 +    assert fake.spec is not None
1591 +    assert fake.spec["video_id"] == video_id
1592 +    assert fake.spec["video_uri"].endswith("source.mp4")
1593 +    # 3 resolved (start, end) pairs from the DB rows
1594 +    assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
1595 +    # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
1596 +    assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
1597 +    # the argv is a `compile` subcommand carrying the spec + done marker
1598 +    argv = fake.calls[0]
1599 +    assert argv[0] == "compile"
1600 +    assert "--spec-uri" in argv and "--done-uri" in argv
1601 +    joined = " ".join(argv)
1602 +    assert "storage.output_root=" in joined # bucket forwarded so the worker mirrors
1603 +    the reel
1604 +
1605 +def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
1606 +    """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
1607 +    coordinates
1608 +    (storage.output_root/backend) ? else the worker can't read the spec or mirror the
1609 +    reel."""
1610 +    db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
1611 +    fake = _FakeCompileClient()
1612 +    _inject_fake(monkeypatch, fake)
1613 +    orch._execute_compile(cfg, db, _job(video_id, seg_ids))
1614 +    joined = " ".join(fake.calls[0])
1615 +    assert f"storage.output_root={bucket}" in joined
1616 +    assert "storage.backend=local" in joined
1617 +
1618 +def test_compile_cloud_failure_returns_failed(compile_env):
1619 +    """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
1620 +    message ? the SPA
1621 +    gets a clean verdict, not a crash."""
1622 +    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1623 +    fake = _FakeCompileClient(returncode=3)
1624 +    _inject_fake(monkeypatch, fake)
1625 +
1626 +    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))

```

```

1623 +     assert out["status"] == "failed"
1624 +     assert "stitch blew up" in out["message"]
1625 +     assert out["video_id"] == video_id
1626 +
1627 +
1628 +def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
1629 +     """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
    _stitch_reel
1630 +     and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
    runs."""
1631 +     db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
1632 +     # flip compile back to the control-plane (the default) ? everything else stays
    cloud-wired
1633 +     cfg.deployment.phase_placement.compile = "control_plane"
1634 +     fake = _FakeCompileClient()
1635 +     _inject_fake(monkeypatch, fake)
1636 +     seen = {}
1637 +
1638 +     def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
1639 +         seen["pairs"] = pairs
1640 +         seen["source"] = source
1641 +         return {"status": "success", "video_id": vid, "download_url":
    f"/api/highlights/{vid}/{name}"}
1642 +
1643 +     monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1644 +
1645 +     out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
1646 +     assert out["status"] == "success"
1647 +     assert fake.calls == [] # NEVER dispatched ? the control-plane stitched in-process
1648 +     assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
1649 +     assert seen["source"].endswith("source.mp4")
1650 +
1651 +
1652 +def test_compile_missing_video_fails_before_dispatch(compile_env):
1653 +     """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
1654 +     db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
1655 +     fake = _FakeCompileClient()
1656 +     _inject_fake(monkeypatch, fake)
1657 +     out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
1658 +     assert out["status"] == "failed"
1659 +     assert fake.calls == [] # never dispatched ? resolve failed first
1660 diff --git a/tests/test_compiler.py b/tests/test_compiler.py
1661 index 1f51622..f3f2ae9 100644
1662 --- a/tests/test_compiler.py
1663 +++ b/tests/test_compiler.py
1664 @@ -158,3 +158,67 @@ def test_compile_raises_when_source_missing(tmp_path):
1665     comp = VideoCompiler(str(tmp_path / "out"))
1666     with pytest.raises(FileNotFoundError):
1667         comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
1668 +
1669 +
1670 +## ----- encoder selection (#521
    ? NVENC)
1671 +
1672 +
1673 +def test_select_encoder_default_is_libx264(tmp_path):
1674 +     """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
    everywhere."""
1675 +     comp = VideoCompiler(str(tmp_path / "out"))
1676 +     args, chosen = comp._select_encoder()
1677 +     assert chosen == "libx264"
1678 +     assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
1679 +
1680 +
1681 +def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):

```

```

1682 +     """h264_nvenc resolves to the hardware encoder args when the probe reports it
runnable (the L4)."""
1683 +     from backend.pipeline.stitcher import compiler as comp_mod
1684 +
1685 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
1686 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1687 +     args, chosen = comp._select_encoder()
1688 +     assert chosen == "h264_nvenc"
1689 +     assert args[:2] == ["-c:v", "h264_nvenc"]
1690 +     assert "-cq" in args # NVENC constant-quality knob (not -crf)
1691 +
1692 +
1693 +def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path,
monkeypatch):
1694 +     """On a box where NVENC can't actually run (CI / no GPU), the requested encoder
falls back to
1695 +     libx264 so a misconfig never nukes the reel."""
1696 +     from backend.pipeline.stitcher import compiler as comp_mod
1697 +
1698 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
1699 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1700 +     args, chosen = comp._select_encoder()
1701 +     assert chosen == "libx264"
1702 +     assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
1703 +
1704 +
1705 +def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
1706 +     """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
1707 +     carries -c:v h264_nvenc, and the concat stays -c copy."""
1708 +     from backend.pipeline.stitcher import compiler as comp_mod
1709 +
1710 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
1711 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1712 +     raw = tmp_path / "raw.mp4"
1713 +     raw.write_bytes(b"x")
1714 +     trims = []
1715 +
1716 +     def fake_run(cmd, *a, **k):
1717 +         if cmd[0] == "ffprobe":
1718 +             return MagicMock(stdout="100.0\n", returncode=0)
1719 +         out = cmd[-1]
1720 +         if isinstance(out, str) and out.endswith(".mp4"):
1721 +             open(out, "wb").write(b"clip")
1722 +             if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
1723 +                 trims.append(cmd)
1724 +             return MagicMock(returncode=0, stdout=b"", stderr=b"")
1725 +
1726 +     with patch.object(subprocess, "run", side_effect=fake_run):
1727 +         comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
1728 +         assert trims, "no trim commands captured"
1729 +         for cmd in trims:
1730 +             assert "h264_nvenc" in cmd # hardware encode on every clip
1731 +             assert "libx264" not in cmd
1732 +diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
1733 +index 5444a34..121ab6e 100644
1734 +--- a/tests/test_compute_backend.py
1735 ++++ b/tests/test_compute_backend.py
1736 +@@ -1100,3 +1100,114 @@ def
test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
1737 +     )
1738 +     assert seen["on_tick"] is heartbeat
1739 +     assert result.video_id == "vidCloud"
1740 +
1741 +

```

```

1742  + # ----- compile dispatch (#521 ? L4
stitch)
1743  +
1744  +
1745  + class _FakeCompileClient(CloudRunJobClient):
1746  +     """Simulates the Cloud Run COMPILE worker: on dispatch, write a reel done-marker
into the local
1747  +     bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-
poll resolves."""
1748  +
1749  +     def __init__(self, *, video_id="vidReel", returncode=0):
1750  +         self.calls: list[dict] = []
1751  +         self.video_id, self.returncode = video_id, returncode
1752  +
1753  +     def run_job(self, job_name, argv, *, region, project=None):
1754  +         self.calls.append({"job": job_name, "argv": list(argv)})
1755  +         done_uri = argv[argv.index("--done-uri") + 1]
1756  +         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
1757  +         with open(done_uri, "w", encoding="utf-8") as f:
1758  +             json.dump(
1759  +                 {
1760  +                     "video_id": self.video_id,
1761  +                     "returncode": self.returncode,
1762  +                     "status": "success" if self.returncode == 0 else "failed",
1763  +                     "download_url": f"/api/highlights/{self.video_id}/reel.mp4",
1764  +                     "reel_uri": f"gs://b/highlights/{self.video_id}/reel.mp4",
1765  +                 },
1766  +                 f,
1767  +             )
1768  +         return "exec-compile"
1769  +
1770  +     def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
1771  +         raise AssertionError("cancel must never fire on the happy path")
1772  +
1773  +
1774  + def test_run_compile_dispatches_and_returns_marker(tmp_path):
1775  +     """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-
polls, and returns
1776  +     the worker's rc + download_url from the marker."""
1777  +     from backend.compute.base import CompileJobSpec
1778  +
1779  +     client = _FakeCompileClient(video_id="vidReel")
1780  +     backend = CloudRunCompute(
1781  +         run_client=client, job_name="rally-worker", region="r", policy=_FAST,
token="tok"
1782  +     )
1783  +     out = str(tmp_path / "bucket")
1784  +     result = backend.run_compile(
1785  +         CompileJobSpec(
1786  +             out_uri=out,
1787  +             spec_uri=f"{out}/_compile/tok/spec.json",
1788  +             video_id="vidReel",
1789  +             output_name="reel.mp4",
1790  +             config_overrides=("storage.output_root=" + out,),
1791  +         )
1792  +     )
1793  +     assert result.returncode == 0
1794  +     assert result.video_id == "vidReel"
1795  +     assert result.report["download_url"] == "/api/highlights/vidReel/reel.mp4"
1796  +     argv = client.calls[0]["argv"]
1797  +     assert argv[0] == "compile"
1798  +     assert argv[argv.index("--spec-uri") + 1] == f"{out}/_compile/tok/spec.json"
1799  +     assert "--done-uri" in argv
1800  +     # the CP-forwarded storage override rides as --set (compile carries NO infer

```

```

container overrides)
1801 +     assert "storage.output_root=" + out in " ".join(argv)
1802 +     assert "deployment.compute_target=local_gpu" not in " ".join(argv)
1803 +
1804 +
1805 +def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1806 +     """A worker stitch failure (rc != 0) is returned as a non-zero ComputeResult (the
dispatcher maps
1807 +     it to a failed job) ? never a false success."""
1808 +     from backend.compute.base import CompileJobSpec
1809 +
1810 +     client = _FakeCompileClient(returncode=3)
1811 +     backend = CloudRunCompute(
1812 +         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1813 +     )
1814 +     out = str(tmp_path / "bk")
1815 +     result = backend.run_compile(
1816 +         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1817 +     )
1818 +     assert result.returncode == 3
1819 +
1820 +
1821 +def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1822 +     """The compile marker lives under _compile/ (not infer's _dispatch/) so a compile +
a detection
1823 +     for the same bucket never collide on a marker."""
1824 +     from backend.compute.base import CompileJobSpec
1825 +
1826 +     client = _FakeCompileClient()
1827 +     backend = CloudRunCompute(
1828 +         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1829 +     )
1830 +     out = str(tmp_path / "bk")
1831 +     backend.run_compile(
1832 +         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1833 +     )
1834 +     done_uri = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
1835 +     assert "/_compile/" in done_uri and "/_dispatch/" not in done_uri
1836 +
1837 +
1838 +def test_run_compile_default_backend_raises_not_implemented():
1839 +     """The ABC default: a backend that can't offload a stitch raises
NotImplementedError (only
1840 +     CloudRunCompute overrides run_compile) ? never a silent no-op."""
1841 +     from backend.compute.base import CompileJobSpec, ComputeBackend, ComputeResult
1842 +
1843 +     class _InProc(ComputeBackend):
1844 +         def run_infer(self, spec, *, on_wait=None):
1845 +             return ComputeResult(returncode=0)
1846 +
1847 +     with pytest.raises(NotImplementedError):
1848 +         _InProc().run_compile(
1849 +             CompileJobSpec(out_uri="x", spec_uri="y", video_id="v",
output_name="r.mp4")
1850 +         )
1851 diff --git a/tests/test_config_deployment.py b/tests/test_config_deployment.py
1852 index 1e4cfa6..5122bcd 100644
1853 --- a/tests/test_config_deployment.py
1854 +++ b/tests/test_config_deployment.py
1855 @@ -70,6 +70,9 @@ def test_default_config_dump_only_adds_deployment_section():
1856     "compute_target": "",
1857     "max_concurrent_remote_jobs": 4,
1858     "remote_poll_max_wait_sec": 3900.0,

```

```

1859 +         # #521: per-phase placement (unset ? legacy default) + the L4 stitch encoder.
1860 +         "phase_placement": {"validation": None, "segment": None, "compile": None},
1861 +         "cloud_stitch_encoder": "h264_nvenc",
1862     }
1863
1864
1865 @@ -397,6 +400,8 @@ def test_save_default_config_includes_deployment(tmp_path):
1866     "compute_target": "",
1867     "max_concurrent_remote_jobs": 4,
1868     "remote_poll_max_wait_sec": 3900.0,
1869 +     "phase_placement": {"validation": None, "segment": None, "compile": None},
1870 +     "cloud_stitch_encoder": "h264_nvenc",
1871     }
1872     cfg = load_config(str(p))
1873     assert cfg.deployment.profile == ""
1874     diff --git a/tests/test_phase_placement.py b/tests/test_phase_placement.py
1875     new file mode 100644
1876     index 0000000..b9cc967
1877     --- /dev/null
1878     +++ b/tests/test_phase_placement.py
1879     @@ -0,0 +1,138 @@
1880     +"""Config-driven phase ? machine placement (#521).
1881     +
1882     +Proves the ``deployment.phase_placement`` knob + ``resolve_phase_placement`` resolver:
1883     the legacy
1884     +defaults reproduce today's placement exactly (validation + compile on the control-
1885     plane, segment on
1886     +the L4 iff compute_target==cloud_run), an explicit knob overrides, the cloud-serving
1887     preset places
1888     +compile on the L4, and the "nobody validates" footgun is rejected at config load.
1889     +"""
1890     +
1891     +import pytest
1892     +
1893     +from backend.config import (
1894     +    PHASE_COMPILE,
1895     +    PHASE_SEGMENT,
1896     +    PHASE_VALIDATION,
1897     +    PLACE_CLOUD_L4,
1898     +    PLACE_CONTROL_PLANE,
1899     +    AppConfig,
1900     +    load_config,
1901     +    resolve_phase_placement,
1902     +)
1903     +
1904     +# ----- legacy defaults
1905     (byte-identical)
1906     +
1907     +
1908     +def test_default_placement_is_todays_behaviour():
1909     +    """No phase_placement block ? validation + compile on the control-plane, segment on
1910     the
1911     +    control-plane (compute_target unset). Byte-identical to pre-#521."""
1912     +    cfg = AppConfig()
1913     +    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
1914     +    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
1915     +    assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CONTROL_PLANE
1916     +
1917     +
1918     +def test_segment_default_follows_compute_target():
1919     +    """The legacy segment default is cloud IFF compute_target==cloud_run ? so a
1920     cloud_run server
1921     +    keeps dispatching DETECT without any phase_placement config."""
1922     +    cloud = AppConfig.model_validate({"deployment": {"compute_target": "cloud_run"}})
1923     +    assert resolve_phase_placement(cloud, PHASE_SEGMENT) == PLACE_CLOUD_L4

```

```

1918 + # validation + compile still default to the control-plane on a cloud_run server
(unset knobs).
1919 + assert resolve_phase_placement(cloud, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
1920 + assert resolve_phase_placement(cloud, PHASE_COMPILE) == PLACE_CONTROL_PLANE
1921 +
1922 + local = AppConfig.model_validate({"deployment": {"compute_target": "local_gpu"}})
1923 + assert resolve_phase_placement(local, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
1924 +
1925 +
1926 +# ----- explicit overrides
1927 +
1928 +
1929 +def test_explicit_knob_overrides_default():
1930 +     """Each phase's explicit knob wins over the legacy default."""
1931 +     cfg = AppConfig.model_validate(
1932 +         {
1933 +             "deployment": {
1934 +                 "compute_target": "cloud_run",
1935 +                 "phase_placement": {
1936 +                     "segment": "control_plane", # force DETECT back on the control-
plane
1937 +                     "compile": "cloud_run_l4", # move stitch to the L4
1938 +                 },
1939 +             }
1940 +         }
1941 +     )
1942 +     assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
1943 +     assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1944 +
1945 +
1946 +def test_resolver_never_raises_on_missing_deployment():
1947 +     """A config-ish object with no deployment/phase_placement degrades to the legacy
default."""
1948 +
1949 +     class _Bare:
1950 +         pass
1951 +
1952 +     assert resolve_phase_placement(_Bare(), PHASE_COMPILE) == PLACE_CONTROL_PLANE
1953 +     assert resolve_phase_placement(_Bare(), PHASE_SEGMENT) == PLACE_CONTROL_PLANE
1954 +
1955 +
1956 +# ----- the cloud-serving
preset
1957 +
1958 +
1959 +def test_cloud_serving_preset_places_compile_on_l4(tmp_path):
1960 +     """The cloud-serving profile preset moves compile to the L4 (the #521 headline)
while keeping
1961 +     validation on the control-plane and segment on the L4 (via compute_target)."""
1962 +     cfg_path = tmp_path / "c.json"
1963 +     cfg_path.write_text('{"deployment": {"profile": "cloud-serving"}}')
1964 +     cfg = load_config(str(cfg_path))
1965 +     assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1966 +     assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
1967 +     assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
1968 +
1969 +
1970 +# ----- the "nobody
validates" guard
1971 +
1972 +
1973 +def test_validation_on_l4_requires_cloud_segment():
1974 +     """validation=cloud_run_l4 with segment on the control-plane would leave NOBODY
validating (the
1975 +     control-plane skips + no worker runs) ? rejected at config load."""

```



```

1976 +     with pytest.raises(Exception, match="phase_placement.validation"):
1977 +         AppConfig.model_validate(
1978 +             {
1979 +                 "deployment": {
1980 +                     "compute_target": "",
1981 +                     "phase_placement": {
1982 +                         "validation": "cloud_run_l4",
1983 +                         "segment": "control_plane",
1984 +                     },
1985 +                 }
1986 +             }
1987 +         )
1988 +
1989 +
1990 +def test_validation_on_l4_ok_with_cloud_segment():
1991 +    """validation=cloud_run_l4 is accepted when segment also runs on the L4
1992 +    (explicit)."""
1993 +    cfg = AppConfig.model_validate(
1994 +        {
1995 +            "deployment": {
1996 +                "phase_placement": {
1997 +                    "validation": "cloud_run_l4",
1998 +                    "segment": "cloud_run_l4",
1999 +                }
2000 +            }
2001 +        )
2002 +    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CLOUD_L4
2003 +
2004 +
2005 +def test_validation_on_l4_ok_when_compute_target_is_cloud_run():
2006 +    """The guard honours the LEGACY segment default: with compute_target=cloud_run the
2007 +    segment runs
2008 +    on the L4 by default, so validation=cloud_run_l4 needs no explicit segment knob."""
2009 +    cfg = AppConfig.model_validate(
2010 +        {
2011 +            "deployment": {
2012 +                "compute_target": "cloud_run",
2013 +                "phase_placement": {"validation": "cloud_run_l4"},
2014 +            }
2015 +        )
2016 +    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CLOUD_L4
2017 +    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
2018 +diff --git a/tests/test_worker.py b/tests/test_worker.py
2019 +index 72b2514..8b72dc1 100644
2020 +--- a/tests/test_worker.py
2021 ++++ b/tests/test_worker.py
2022 +@@ -6,6 +6,7 @@ on the droplet against MinIO.
2023 +    """
2024 +
2025 +    import json
2026 +    import os
2027 +
2028 +    import pytest
2029 +
2030 +    @@ -943,3 +944,177 @@ def
test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
2031 +        bad = tmp_path / "not_a_video.mp4"
2032 +        bad.write_bytes(b"NOTAVIDEO")
2033 +        assert wmain._probe_video_fps_width(str(bad)) is None
2034 +
2035 +
2036 +    +# ----- compile subcommand (#521 ?
L4 stitch)

```

```

2037 +
2038 +
2039 +def test_parser_compile_accepts_spec_and_done_uris():
2040 +    args = wmain.build_parser().parse_args(
2041 +        [
2042 +            "compile",
2043 +            "--spec-uri",
2044 +            "gs://b/_compile/tok/spec.json",
2045 +            "--out-uri",
2046 +            "gs://b",
2047 +            "--done-uri",
2048 +            "gs://b/_compile/tok.done",
2049 +            "--set",
2050 +            "storage.output_root=gs://b",
2051 +        ]
2052 +    )
2053 +    assert args.cmd == "compile"
2054 +    assert args.spec_uri == "gs://b/_compile/tok/spec.json"
2055 +    assert args.done_uri == "gs://b/_compile/tok.done"
2056 +    assert args.set == ["storage.output_root=gs://b"]
2057 +
2058 +
2059 +def _write_compile_spec(bucket, video_id, source, name, encoder="h264_nvenc"):
2060 +    spec = {
2061 +        "video_id": video_id,
2062 +        "video_uri": str(source),
2063 +        "output_name": name,
2064 +        "locale": None,
2065 +        "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
2066 +        "stitching": {"encoder": encoder, "watermark": {"enabled": False}},
2067 +    }
2068 +    spec_dir = os.path.join(str(bucket), "_compile", "tok")
2069 +    os.makedirs(spec_dir, exist_ok=True)
2070 +    spec_uri = os.path.join(spec_dir, "spec.json")
2071 +    with open(spec_uri, "w", encoding="utf-8") as f:
2072 +        json.dump(spec, f)
2073 +    return spec_uri
2074 +
2075 +
2076 +def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
2077 +    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core
2078 +    (VideoCompiler stubbed so
2079 +    no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
2080 +    bucket-poll."""
2081 +    import backend.orchestration as orch
2082 +
2083 +    bucket = tmp_path / "bucket"
2084 +    bucket.mkdir()
2085 +    source = tmp_path / "src.mp4"
2086 +    source.write_bytes(b"SRC")
2087 +    spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
2088 +    done = str(bucket / "_compile" / "tok.done")
2089 +
2090 +    seen = {}
2091 +
2092 +    def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name,
2093 +        locale, notify):
2094 +        seen["encoder"] = stitching.encoder
2095 +        seen["pairs"] = time_pairs
2096 +        seen["source"] = source_ref
2097 +        notify("stitching")
2098 +        return {
2099 +            "status": "success",
2100 +            "video_id": video_id,
2101 +            "download_url": f"/api/highlights/{video_id}/{out_name}",

```

```

2099 +         "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
2100 +     }
2101 +
2102 + monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
2103 +
2104 + cfg = tmp_path / "c.json"
2105 + cfg.write_text('{"storage": {"backend": "local"}}')
2106 + rc = wmain.cmd_compile(
2107 +     wmain.build_parser().parse_args(
2108 +         [
2109 +             "compile",
2110 +             "--spec-uri",
2111 +             spec_uri,
2112 +             "--out-uri",
2113 +             str(bucket),
2114 +             "--done-uri",
2115 +             done,
2116 +             "--config",
2117 +             str(cfg),
2118 +             "--scratch",
2119 +             str(tmp_path / "s"),
2120 +             "--set",
2121 +             f"storage.output_root={bucket}",
2122 +         ]
2123 +     )
2124 + )
2125 + assert rc == 0
2126 + # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
2127 + assert seen["encoder"] == "h264_nvenc"
2128 + assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
2129 + assert seen["source"] == str(source)
2130 + marker = json.loads(open(done, encoding="utf-8").read())
2131 + assert marker["returncode"] == 0 and marker["status"] == "success"
2132 + assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
2133 +
2134 +
2135 +def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
2136 +     """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the
2137 + dispatcher's poll
2138 +     terminates and the SPA sees a clean failure."""
2139 +     import backend.orchestration as orch
2140 +
2141 +     bucket = tmp_path / "bucket"
2142 +     bucket.mkdir()
2143 +     source = tmp_path / "src.mp4"
2144 +     source.write_bytes(b"SRC")
2145 +     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
2146 +     done = str(bucket / "_compile" / "tok.done")
2147 +
2148 +     monkeypatch.setattr(
2149 +         orch,
2150 +         "_stitch_reel",
2151 +         lambda *a, **k: {"status": "failed", "message": "ffmpeg blew up", "video_id":
2152 + "vidReel"},
2153 +     )
2154 +     cfg = tmp_path / "c.json"
2155 +     cfg.write_text('{"storage": {"backend": "local"}}')
2156 +     rc = wmain.cmd_compile(
2157 +         wmain.build_parser().parse_args(
2158 +             [
2159 +                 "compile",
2160 +                 "--spec-uri",
2161 +                 spec_uri,
2162 +                 "--out-uri",
2163 +                 str(bucket),

```

```

2162 +         "--done-uri",
2163 +         done,
2164 +         "--config",
2165 +         str(cfg),
2166 +         "--scratch",
2167 +         str(tmp_path / "s"),
2168 +     ]
2169 + )
2170 + )
2171 + assert rc == 3
2172 + marker = json.loads(open(done, encoding="utf-8").read())
2173 + assert marker["returncode"] == 3 and marker["status"] == "failed"
2174 +
2175 +
2176 +def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
2177 +     """DEFAULT-OFF: a direct-CLI stitch with no --done-uri writes no marker."""
2178 +     import backend.orchestration as orch
2179 +
2180 +     bucket = tmp_path / "bucket"
2181 +     bucket.mkdir()
2182 +     source = tmp_path / "src.mp4"
2183 +     source.write_bytes(b"SRC")
2184 +     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
2185 +
2186 +     monkeypatch.setattr(
2187 +         orch, "_stitch_reel", lambda *a, **k: {"status": "success", "video_id":
"vidReel"}
2188 +     )
2189 +     cfg = tmp_path / "c.json"
2190 +     cfg.write_text('{"storage": {"backend": "local"}}')
2191 +     rc = wmain.cmd_compile(
2192 +         wmain.build_parser().parse_args(
2193 +             [
2194 +                 "compile",
2195 +                 "--spec-uri",
2196 +                 spec_uri,
2197 +                 "--out-uri",
2198 +                 str(bucket),
2199 +                 "--scratch",
2200 +                 str(tmp_path / "s"),
2201 +                 "--config",
2202 +                 str(cfg),
2203 +             ]
2204 +         )
2205 +     )
2206 +     assert rc == 0
2207 +     assert not (bucket / "_compile" / "tok.done").exists()

```

12. assistant (2026-07-08T06:05:45.301Z)

I have the full diff. Now let me examine the production source to verify real failure modes the tests don't cover. Let me read the key models and orchestration functions.

13. user (2026-07-08T06:05:46.964Z)

```

280     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282     # Matches gemini_refine's --clip-scale-width default.
283     ai_chunk_scale_width: int = 1280
284     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the

```

```

Gemini
288     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289     skip_ai_handoff: bool = False
290     # Optional debug knob: trim every video to the first N seconds before processing.
291     debug_duration: Optional[float] = None
292
293     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295     fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297     @field_validator("default_segmenter")
298     @classmethod
299     def segmenter_must_be_registered(cls, v: str) -> str:
300         # Lazy import: the segmenter registry imports modules that may import
301         # config, so we resolve it at validation time rather than at module load.
302         from backend.pipeline.segmenters.base import segmenter_registry
303
304         if segmenter_registry.get(v) is None:
305             available = sorted(segmenter_registry.list_available().keys())
306             raise ValueError(
307                 f"Segmenter '{v}' is not registered. Available: {'', ' '.join(available)}"
308             )
309         return v
310
311     @model_validator(mode="after")
312     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314         # step makes `while t < duration: t += step` loop forever, growing memory before
any
315         # GPU work. Reject the bad config at load time instead.
316         if self.chunk_overlap >= self.chunk_duration:
317             raise ValueError(
318                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                 f"({self.chunk_duration}); otherwise chunking never advances."
320             )
321         return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
329         output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
341         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans" # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         ) # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True # faint backing box behind the text for legibility

```

```

348     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
349     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350         "bottom-right"
351     )
352
353
354     class SdRenditionConfig(BaseModel):
355         """Optional low-resolution reel encode profile.
356
357         This is always available to callers that explicitly request it; the knobs stay
358         configurable because different hosts may prefer a smaller 360p file or a more
359         readable 480p file. The original rendition remains the historical source-size
360         encode path and is intentionally not configurable through this profile.
361         """
362
363         max_width: int = Field(default=854, ge=16)
364         max_height: int = Field(default=480, ge=16)
365         crf: int = Field(default=30, ge=0, le=51)
366         audio_bitrate: str = "96k"
367         video_preset: str = "superfast"
368         pix_fmt: str = "yuv420p"
369
370
371     class StitchingRenditionsConfig(BaseModel):
372         sd: SdRenditionConfig = Field(default_factory=SdRenditionConfig)
373
374
375     class StitchingConfig(BaseModel):
376         begin_padding: float = 0.5
377         end_padding: float = 0.5
378         use_cache: bool = False
379         min_shot_count: int = 1
380         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
381         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
382         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
383         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
384         # local detector over-fires and its false positives are mostly SHORT (motion blips,
385         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
386         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
387         min_rally_duration: float = Field(default=0.0, ge=0.0)
388         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
389         renditions: StitchingRenditionsConfig = Field(
390             default_factory=StitchingRenditionsConfig
391         )
392
393
394     class LoggingConfig(BaseModel):
395         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
396
397         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
398         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
399         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises

```

14. user (2026-07-08T06:05:47.589Z)

```

1180         out_name,
1181     )
1182     local_src = ""
1183     try:
1184         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local

```

```

path). Resolve to
1185         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1186         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1187         local_src = storage.acquire_local_input(
1188             video["filepath"], getattr(cfg, "storage", None)
1189         )
1190         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1191         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1192         with _LOCAL_COMPUTE_LANE:
1193             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1194         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1195             logger.error(
1196                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1197             )
1198             return {
1199                 "status": "failed",
1200                 "message": f"Highlight stitching failed: {e}",
1201                 "video_id": video_id,
1202             }
1203         finally:
1204             if local_src:
1205                 storage.cleanup_acquired_local_input(
1206                     video["filepath"], local_src, getattr(cfg, "storage", None)
1207                 )
1208
1209         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1210         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1211         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1212         # still falls back to the local file on the same instance).
1213         reel_uri = reel_object_uri(cfg, video_id, out_name)
1214         if reel_uri:
1215             try:
1216                 notify("uploading reel")
1217                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1218                 # project/client) is used, not a fresh default.
1219                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1220             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1221                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1222
1223         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1224         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1225         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1226         notify("finished")
1227         return {
1228             "status": "success",
1229             "filepath": out_path,
1230             "download_url": download_url,
1231             "video_id": video_id,
1232             "filename": out_name,
1233             "rendition": rendition,
1234         }
1235

```

The function at line 1180 uses `video["filepath"]` and returns `rendition` ? that's not the `\_stitch\_reel` from the diff. Let me locate the actual functions in the current tree.

16. user (2026-07-08T06:06:05.488Z)

```
1103: def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1104:     if rendition != "sd":
1106:         renditions = (
1107:             stitching.get("renditions")
1109:             else getattr(stitching, "renditions", None)
1111:         if isinstance(renditions, dict):
1112:             return renditions.get("sd")
1113:         return getattr(renditions, "sd", None)
1130: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1142:     rendition = str(params.get("rendition") or "original")
1143:     if rendition not in {"original", "sd"}:
1146:         "message": f"Unsupported compile rendition: {rendition}",
1165:     encode_profile = _rendition_encode_profile(stitching, rendition)
1177:     "[compile] stitching video_id=%s rendition=%s output=%s",
1179:     rendition,
1233:     "rendition": rendition,
```

17. user (2026-07-08T06:06:07.893Z)

```
Exit code 1
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
---HEAD contains dac6736?---
```

18. assistant (2026-07-08T06:06:48.277Z)

The working tree is PRE-change (`dac6736` is not in HEAD). I must read the post-change files at commit `dac6736`. Let me extract them.

19. user (2026-07-08T06:06:54.248Z)

done

20. user (2026-07-08T06:06:55.967Z)

```
44: from backend.pipeline.stitcher.watermark_i18n import localize_watermark
199: def _cloud_available(cfg) -> bool:
218: def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
253:     return None
256:     return None
266:     return None
376:     return None
379:     return None
383:     return None
386:     return None
400:     return None
408:     return None
422:     return None
450: def _maybe_dispatch_remote(
525:     return None # explicit "run on my machine" ? in-process regardless of the server's
target
547:     return None # default-OFF / placed on control_plane ? the in-process segmenter
below
1087:     return None
1100: def _resolve_compile(
1149: def reel_object_uri(cfg, video_id: str, name: str) -> str:
```



```

1159: def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1174:     return None
1178:     return None
1190: def _stitch_reel(
1214:     localized_watermark = localize_watermark(stitching.watermark, locale)
1279: def _dispatch_compile_remote(
1423: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

21. user (2026-07-08T06:06:59.987Z)

```

450     def _maybe_dispatch_remote(
451         cfg,
452         db,
453         req: "ProcessRequest",
454         video_id: str,
455         seg_name: str,
456         val_results,
457         play_wm: int,
458         cand_wm: int,
459         notify,
460     ) -> Optional[Tuple[Dict[str, Any], bool]]:
461         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
462         ``deployment.compute_target
463         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
464         intervals into
465         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
466         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
467         the cloud job
468         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
469         observability report
470         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
471         process segmenter
472         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
473         non-``cloud_run`` target, so
474         the in-process path stays byte-identical)."""
475         from backend.compute import ComputeJobSpec, get_compute_backend
476
477         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
478             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
479             (id > play_wm);
480             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
481             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
482             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
483             only when the
484             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
485             rc / ingest
486             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
487             records the
488             # intended backend, `requested` the picker choice, `dispatched` whether the
489             remote job ran.
490             return (
491                 {
492                     "video_id": video_id,
493                     "status": "failed",
494                     "message": msg,
495                     "results": [r.model_dump() for r in val_results],
496                     "play_segments": [],
497                     "compute": {
498                         "path": "cloud_run" if ran else "not_run",
499                         "requested": req.compute,
500                         "target": "cloud_run",
501                         "dispatched": ran,
502                     },
503                 },
504             ),

```

```

494         ran,
495     )
496
497     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
499     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500     choice = (req.compute or "").strip().lower()
501     if choice and choice not in ("local", "cloud"):
502         logger.warning(
503             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
504             req.compute,
505         )
506         choice = ""
507     if choice == "local":
508         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
510         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
511         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
512         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
513         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
514         # below exactly as before (the weights-present check is the escape hatch).
515         if (
516             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
517             and _cloud_available(cfg)
518             and not _local_detector_weights_present(cfg, seg_name)
519         ):
520             return _failed(
521                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
522                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'." ,
523                 False,
524             )
525             return None # explicit "run on my machine" ? in-process regardless of the
server's target
526         if choice == "cloud":
527             if not _cloud_available(cfg):
528                 return _failed(
529                     "cloud-serving was requested but this server isn't configured for it "
530                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
531                     False,
532                 )
533             compute = get_compute_backend(cfg, target_override="cloud_run")
534         else:
535             # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
536             # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
537             # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
538             # control-plane, or onto the L4 on a default-local box wired for cloud).
539             if (
540                 resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
541                 and _cloud_available(cfg)

```

```

542         ):
543             compute = get_compute_backend(cfg, target_override="cloud_run")
544         else:
545             compute = None
546     if compute is None:
547         return None # default-OFF / placed on control_plane ? the in-process segmenter
below
548
549     storage_cfg = getattr(cfg, "storage", None)
550
551     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
552     try:
553         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
554     except ValueError as e:
555         return _failed(
556             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
557         )
558     if input_ref.backend == "local":
559         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
560         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
561         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
562         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
563         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
564         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
565         # default, while a bare/local-looking root can't stage no matter what
input_backend says
566         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
567         # gives us nothing to build the staged URI under).
568         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
569         try:
570             root_backend = (
571                 storage.parse_uri(input_root).backend if input_root else "local"
572             )
573         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
574             root_backend = "local"
575         if root_backend == "local":
576             return _failed(
577                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
578                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
579                 "Run job.",
580                 False,
581             )
582         local_src = input_ref.key # resolved local path of the upload
583         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
584         try:
585             notify("staging upload to bucket")
586             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
587             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
588             storage.put(
589                 local_src,
590                 staged_uri,

```

```

591         config=(
592             storage_cfg.model_dump() if storage_cfg is not None else None
593         ),
594     )
595     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
596         logger.error(
597             "[API] cloud-serving: failed to stage %s -> %s: %s",
598             local_src,
599             staged_uri,
600             e,
601             exc_info=True,
602         )
603         return _failed(
604             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
605             False,
606         )
607         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
608         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
609         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
610         req.video_path = staged_uri
611         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
612         db.add_video(
613             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
614         )
615         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
616         if not out_root:
617             return _failed(
618                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
619                 "? that's where the Cloud Run job writes the result.",
620                 False,
621             )
622
623         notify("dispatching (cloud_run)")
624         logger.info(
625             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626             req.video_path,
627             out_root,
628         )
629         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with

```

22. assistant (2026-07-08T06:07:19.995Z)

This confirms a divergence risk. Let me verify whether `\_cloud\_available` can be True while `get\_compute\_backend(target\_override="cloud\_run")` returns None, and read `\_cloud\_available`, `\_stitch\_reel`, `\_storage\_settings`.

23. user (2026-07-08T06:07:23.978Z)

```

199     def _cloud_available(cfg) -> bool:
200         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
201
202         True when the control plane is wired for Cloud Run: either the configured default
target is
203         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
204         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)

```

```

205         are present so a forced per-request override can actually dispatch + collect a
result.
206
207         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
208         deployment = getattr(cfg, "deployment", None)
209         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
210             return True
211         has_job = bool(
212             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
213         )
214         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
215         return bool(has_job and out_root)
216
217
218     def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
219         """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
220         ``maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
221         ``execute_processing`` can
222         decide whether to DELEGATE validation to the worker
223         (``phase_placement.validation``). A per-request
224         picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
225         cloud iff
226         ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
227         pre-dispatch reject
228         (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
229         remotely."""
230         choice = (req_compute or "").strip().lower()
231         if choice == "local":
232             return False
233         if choice == "cloud":
234             return _cloud_available(cfg)
235         return (
236             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
237             and _cloud_available(cfg)
238         )
239
240
241     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
242         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
243         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
244         comes
245         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
246         "starting"
247         (queue/cold-start), so the very first tick already shows movement."""
248         minutes = int(elapsed_sec) // 60
249         if minutes < 1:
250             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
251         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
252
253
254     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
255         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
256         gs://
257         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
258         keep
259         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
260         no
261         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
262         if getattr(input_ref, "backend", "") != "gcs":
263             return None

```

24. user (2026-07-08T06:07:24.595Z)

```

1149     def reel_object_uri(cfg, video_id: str, name: str) -> str:

```

```

1150         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1151         (#463), or "" when
1152         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1153         compile
1154         mirror-upload and the /api/highlights redirect:
1155         ``<output_root>/highlights/<video_id>/<name>``. """
1156         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1157         if not root or "://" not in root:
1158             return ""
1159         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1160
1161     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1162         """The active storage settings dict (per-backend config for the storage layer ? S3
1163         endpoint/region,
1164         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1165         default backend
1166         isn't silently constructed as a fresh default. """
1167         sc = getattr(cfg, "storage", None)
1168         return sc.model_dump() if sc is not None else None
1169
1170     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1171         """A short-lived signed URL for a reel mirrored to the output bucket, so
1172         /api/highlights can
1173         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1174         ephemeral local
1175         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1176         config for BOTH
1177         the existence check and the signing (extracted here to keep backend/main.py lean ?
1178         P23/P24). """
1179         reel_uri = reel_object_uri(cfg, video_id, name)
1180         if not reel_uri:
1181             return None
1182         settings = _storage_settings(cfg)
1183         if storage.exists(reel_uri, config=settings):
1184             return storage.signed_url(reel_uri, config=settings, method="GET")
1185         return None
1186
1187     def _compile_wait_message(elapsed_sec: float) -> str:
1188         """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1189         ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start. """
1190         minutes = int(elapsed_sec) // 60
1191         if minutes < 1:
1192             return "stitching on cloud GPU (NVENC) ? starting up"
1193         return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1194
1195     def _stitch_reel(
1196         cfg,
1197         stitching,
1198         video_id: str,
1199         source_ref: str,
1200         time_pairs: List[Tuple[float, float]],
1201         out_name: str,
1202         locale: Optional[str],
1203         notify,
1204     ) -> Dict[str, Any]:
1205         """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
1206         L4 worker
1207         (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
1208         ``_resolve_compile``), so this
1209         needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
1210         VideoCompiler

```

```

1203         with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204         trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205         returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206         ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209         output_dir = (
1210             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211         )
1212         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213         # unknown/absent locale keeps the configured default unchanged.
1214         localized_watermark = localize_watermark(stitching.watermark, locale)
1215         compiler = _main.VideoCompiler(
1216             output_dir,
1217             begin_padding=stitching.begin_padding,
1218             end_padding=stitching.end_padding,
1219             watermark=localized_watermark,
1220             encoder=getattr(stitching, "encoder", "libx264"),
1221         )
1222
1223         notify("stitching")
1224         local_src = ""
1225         try:
1226             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228             # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229             local_src = storage.acquire_local_input(
1230                 source_ref, getattr(cfg, "storage", None)
1231             )
1232             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233             # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234             with _LOCAL_COMPUTE_LANE:
1235                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236             except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237                 logger.error(
1238                     "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239                 )
1240             return {
1241                 "status": "failed",
1242                 "message": f"Highlight stitching failed: {e}",
1243                 "video_id": video_id,
1244             }
1245         finally:
1246             if local_src:
1247                 storage.cleanup_acquired_local_input(
1248                     source_ref, local_src, getattr(cfg, "storage", None)
1249                 )
1250
1251         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1252         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1253         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch

```

```

(the endpoint
1254         # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1255         # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1256         reel_uri = reel_object_uri(cfg, video_id, out_name)
1257         if reel_uri:
1258             try:
1259                 notify("uploading reel")
1260                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1261                 # project/client) is used, not a fresh default.
1262                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1263             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1264                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1265
1266         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1267         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1268         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1269         notify("finished")
1270         return {
1271             "status": "success",
1272             "filepath": out_path,
1273             "download_url": download_url,
1274             "video_id": video_id,
1275             "reel_uri": reel_uri or "",
1276         }
1277
1278
1279     def _dispatch_compile_remote(
1280         cfg,
1281         db,
1282         video_id: str,
1283         segment_ids: List[int],
1284         output_filename: str,
1285         locale: Optional[str],
1286         notify,
1287     ) -> Dict[str, Any]:
1288         """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The

```

25. assistant (2026-07-08T06:07:42.106Z)

Let me check ``get_compute_backend`` to see if it can return `None` when ``_cloud_available`` is `True` (the validation-delegation divergence), and read the post-change ``_dispatch_compile_remote`` + ``_execute_compile``.

26. user (2026-07-08T06:07:44.849Z)

"""Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.

COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and ``None`` otherwise ? ``None`` means "run the worker's existing in-process path" (today's behaviour, byte identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` / ``cpu_mock`` nothing changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when needed).

"""

```
from __future__ import annotations
```

```
import dataclasses
```



```

import os
from typing import Any, Optional

from backend.compute.base import (
    CompileJobSpec,
    ComputeBackend,
    ComputeJobSpec,
    ComputeResult,
    RemoteExecutionFailed,
)
from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend",
    "ComputeJobSpec",
    "CompileJobSpec",
    "ComputeResult",
    "RemoteExecutionFailed",
    "get_compute_backend",
]

def _storage_dict(config: Any) -> dict:
    storage = getattr(config, "storage", None)
    if storage is None:
        return {}
    if hasattr(storage, "model_dump"):
        return storage.model_dump() # type: ignore[no-any-return]
    return dict(storage) if isinstance(storage, dict) else {}

def get_compute_backend(
    config: Any,
    *,
    run_client: Optional[Any] = None,
    policy: ExecutionPolicy = DEFAULT_POLICY,
    token: Optional[str] = None,
    target_override: Optional[str] = None,
) -> Optional[ComputeBackend]:
    """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to run
    in-process (the default-OFF path).

    Only ``compute_target == "cloud_run"`` returns a non-None backend (a ``CloudRunCompute``).
    Its
    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
    ``CLOUD_RUN_REGION``
    / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is injectable
    (tests
    pass a fake; production defaults to the real ``google-cloud-run`` client, owner-verified on
    M7).

    ``target_override`` lets a single request force a target (the SPA compute-picker maps a
    per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating the
    server's configured default. ``None`` falls back to ``config.deployment.compute_target``.
    """
    target = (
        target_override
        if target_override is not None
        else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
    )
    if target != "cloud_run":
        return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

    # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually selected.
    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

```

```

# Derive the poll budget from config UNLESS the caller injected an explicit policy ?
# deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout so a
# slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the old
# 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps an
# explicitly-passed policy (tests, callers with bespoke budgets) untouched.
if policy is DEFAULT_POLICY:
    max_wait = getattr(
        getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
    )
    if max_wait:
        policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))

job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
project = os.environ.get("GOOGLE_CLOUD_PROJECT")
client = run_client if run_client is not None else GoogleCloudRunJobClient()
return CloudRunCompute(
    run_client=client,
    job_name=job_name,
    region=region,
    project=project,
    storage_config=_storage_dict(config),
    policy=policy,
    token=token,
)

```

27. user (2026-07-08T06:07:46.349Z)

```

1289         control-plane owns the DB, so it RESOLVES the reel's segments here
(`_resolve_compile`), stages a
1290         small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1291         to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1292         done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
1293         worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1294         ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1295         from backend.compute import CompileJobSpec, get_compute_backend
1296
1297         notify("resolving")
1298         try:
1299             video, kept, out_name = _resolve_compile(
1300                 db, cfg, video_id, segment_ids, output_filename
1301             )
1302         except _CompileError as e:
1303             return {"status": "failed", "message": e.detail, "video_id": video_id}
1304
1305         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1306         storage_cfg = getattr(cfg, "storage", None)
1307         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1308         compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1309         if compute is None:
1310             # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
1311             # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
1312             # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1313             logger.warning(
1314                 "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1315                 "stitching in-process on the control-plane instead.",
1316                 out_root,

```

```

1317         )
1318         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1319         return _stitch_reel(
1320             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
1321             notify
1322         )
1323         # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
1324         # on the wire);
1325         # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
1326         # to the L4's
1327         # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
1328         # the reel.
1329         stitch_dump = stitching.model_dump()
1330         stitch_dump["encoder"] = getattr(
1331             getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1332         )
1333         spec_payload = {
1334             "video_id": video_id,
1335             "video_uri": video["filepath"],
1336             "output_name": out_name,
1337             "locale": locale,
1338             "time_pairs": [
1339                 [float(s["start_time"]), float(s["end_time"])] for s in kept
1340             ],
1341             "stitching": stitch_dump,
1342         }
1343         token = uuid.uuid4().hex
1344         spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1345         try:
1346             notify("staging compile spec")
1347             with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1348                 local_spec = os.path.join(td, "spec.json")
1349                 with open(local_spec, "w", encoding="utf-8") as f:
1350                     json.dump(spec_payload, f)
1351                 storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1352         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
1353             not a crash
1354             logger.error(
1355                 "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
1356                 exc_info=True
1357             )
1358             return {
1359                 "status": "failed",
1360                 "message": f"cloud compile: could not stage the compile spec: {e}",
1361                 "video_id": video_id,
1362             }
1363         # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
1364         # spec, mirror the
1365         # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
1366         # knobs for detect.
1367         overrides = [f"storage.output_root={out_root}"]
1368         backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1369         if backend_name:
1370             overrides.append(f"storage.backend={backend_name}")
1371         spec = CompileJobSpec(
1372             out_uri=out_root,
1373             spec_uri=spec_uri,
1374             video_id=video_id,
1375             output_name=out_name,
1376             config_overrides=tuple(overrides),
1377         )
1378         notify("dispatching stitch (cloud_run)")

```

```

1374     logger.info(
1375         "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
1376         video_id,
1377         out_root,
1378         len(kept),
1379     )
1380     try:
1381         result = compute.run_compile(
1382             spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1383         )
1384     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1385         logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1386         return {
1387             "status": "failed",
1388             "message": f"cloud compile dispatch error: {e}",
1389             "video_id": video_id,
1390         }
1391     if result.returncode != 0:
1392         msg = str(
1393             result.report.get("message")
1394             or f"cloud compile job failed (returncode {result.returncode})."
1395         )
1396         return {"status": "failed", "message": msg, "video_id": video_id}
1397
1398     download_url = str(
1399         result.report.get("download_url")
1400         or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1401     )
1402     logger.info(
1403         "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
1404         result.execution,
1405         result.outputs_uri,
1406         video_id,
1407     )
1408     notify("finished")
1409     return {
1410         "status": "success",
1411         "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1412         "download_url": download_url,
1413         "video_id": video_id,
1414         "compute": {
1415             "path": "cloud_run",
1416             "execution": result.execution,
1417             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
1418             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
1419         },
1420     }
1421
1422
1423     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1424         """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1425         the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
1426         result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
1427         a clean verdict instead of a worker-crash error.
1428
1429         #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
1430         dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1431         (``dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the

```

```

1432         2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
1433         notify = progress or (lambda stage: None)
1434         params = json.loads(job.get("params") or "{}")
1435         video_id = job.get("video_id") or ""
1436         segment_ids = params.get("segment_ids") or []
1437         locale = params.get("locale")
1438         output_filename = params.get("output_filename", "")
1439
1440         if (
1441             resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1442             and _cloud_available(cfg)
1443         ):
1444             return _dispatch_compile_remote(
1445                 cfg, db, video_id, segment_ids, output_filename, locale, notify
1446             )
1447
1448         notify("resolving")
1449         try:
1450             video, kept, out_name = _resolve_compile(
1451                 db, cfg, video_id, segment_ids, output_filename
1452             )
1453         except _CompileError as e:
1454             return {"status": "failed", "message": e.detail, "video_id": video_id}
1455
1456         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1457         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1458         return _stitch_reel(
1459             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
1460             notify
1461         )

```

28. assistant (2026-07-08T06:09:26.739Z)

Let me read the watermark localize path (a listed candidate) and check the worker `cmd\_compile` reconstruction against it.

29. user (2026-07-08T06:09:29.211Z)

```

1         """Localized reel-watermark text + font selection.
2
3         The SPA passes the UI locale to ``POST /api/compile``; this maps it to a localized
branding string
4         and a script-appropriate bundled font, so a Kannada/Hindi/? user's reel carries the
watermark in
5         their language. The brand token ("KhelSutra.guru") stays Latin; only the surrounding
phrase localizes.
6
7         Fonts (``backend/assets/fonts``):
8         - en/es/da/id (Latin) ? the configured default (bundled RobotoSlab, Apache-2.0)
9         - kn ? Noto Sans Kannada · hi ? Noto Sans Devanagari (OFL-1.1, bundled ? the D7 OFL-
for-media
10         carve-out; FRIENDLY_AND_MULTILINGUAL $4 anticipated this).
11
12         Honest fallbacks: an unknown/missing locale (or a locale with no entry, incl. ``en``)
returns the
13         watermark UNCHANGED, so existing/default behavior is preserved. If a script font file is
somehow
14         absent, ``font_path`` is left at the default (RobotoSlab) rather than pointing at a
missing file ?
15         and the compiler already retries the encode without the watermark if drawtext can't
render, so a
16         font gap can never nuke the reel. Kept config-package-agnostic (operates on a dict).
17         """

```

```

18
19     from pathlib import Path
20     from typing import Any, Dict, Optional
21
22     # This module is backend/pipeline/stitcher/ ? parents[2] == backend/ (matches
compiler.py).
23     _FONTS_DIR = Path(__file__).resolve().parents[2] / "assets" / "fonts"
24
25     # Localized watermark phrase per locale. AI-drafted ? native-speaker review pending
(same posture as
26     # the SPA catalogs). `en` is intentionally absent: it falls through to the configured
default text.
27     WATERMARK_TEXT: Dict[str, str] = {
28         "es": "Generado con KhelSutra.guru",
29         "da": "Lavet med KhelSutra.guru",
30         "id": "Dibuat dengan KhelSutra.guru",
31         "kn": "KhelSutra.guru ??? ????",
32         "hi": "KhelSutra.guru ?? ???? ??",
33     }
34
35     # Script-specific bundled fonts ? only where the Latin default (RobotoSlab) can't render
the script.
36     # Latin locales (es/da/id) localize the TEXT but keep the default font.
37     FONT_FILE_BY_LOCALE: Dict[str, str] = {
38         "kn": "NotoSansKannada-Variable.ttf",
39         "hi": "NotoSansDevanagari-Variable.ttf",
40     }
41
42
43     def _base_locale(locale: Optional[str]) -> str:
44         """Normalize a BCP-47 tag to its base subtag: 'kn-IN'/'kn_IN'/'KN' ? 'kn'."""
45         return (locale or "").strip().lower().replace("_", "-").split("-")[0]
46
47
48     def localize_watermark(watermark: Any, locale: Optional[str]) -> Any:
49         """Return a watermark dict with ``text`` (and ``font_path`` for non-Latin scripts)
localized for
50         ``locale``. Returns the input UNCHANGED for ``en`` / unknown / missing locale
(default behavior).
51
52         Accepts a ``WatermarkConfig`` model or a plain dict; returns a dict when it
localizes (the shape
53         ``VideoCompiler`` already accepts), or the original object untouched when there's
nothing to do.
54         """
55         base = _base_locale(locale)
56         if base not in WATERMARK_TEXT:
57             return watermark # en / unsupported / missing ? configured default, unchanged
58
59         if hasattr(watermark, "model_dump"):
60             wm: Dict[str, Any] = watermark.model_dump()
61         elif isinstance(watermark, dict):
62             wm = dict(watermark)
63         else:
64             return watermark # unknown shape ? don't risk mangling it
65
66         wm["text"] = WATERMARK_TEXT[base]
67         font_file = FONT_FILE_BY_LOCALE.get(base)
68         if font_file:
69             font_path = _FONTS_DIR / font_file
70             if font_path.exists():
71                 wm["font_path"] = str(font_path)
72             # else: leave the configured/default font_path (RobotoSlab) ? a graceful Latin
fallback.
73         return wm

```

30. assistant (2026-07-08T06:10:36.784Z)

Let me read `\_resolve\_compile` to check whether `min\_rally\_duration`/`min\_shot\_count` filtering is consistent between the cloud spec path and local path.

31. user (2026-07-08T06:10:40.360Z)

```

1100 def _resolve_compile(
1101     db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1102 ):
1103     """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
1104     resolve, since a
1105     row could change between submit and run). Verifies the video exists, the requested
1106     segments match
1107     and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
1108     shot_count are exempt
1109     so manual/legacy picks can't silently vanish), and the output name is traversal-
1110     safe. Raises
1111     ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
1112     safe_out_name)``."""
1113     video = db.get_video(video_id)
1114     if not video:
1115         raise _CompileError(404, "Indexed video record not found.")
1116     # Reject path traversal / absolute names (os.path.join would otherwise write
1117     anywhere on disk).
1118     try:
1119         out_name = safe_output_filename(output_filename)
1120     except ValueError as e:
1121         raise _CompileError(400, str(e)) from e
1122
1123     all_segments = db.query_play_segments(video_id, {})
1124     matched = [s for s in all_segments if s["id"] in segment_ids]
1125     if not matched:
1126         raise _CompileError(400, "No matching play segments found to stitch.")
1127
1128     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1129     kept = []
1130     for s in matched:
1131         meta = s.get("metadata") or {}
1132         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1133         try:
1134             if sc is not None and int(sc) < stitching.min_shot_count:
1135                 continue
1136         except (TypeError, ValueError):
1137             pass # unparseable count -> treat as unknown, keep
1138         kept.append(s)
1139     if len(kept) < len(matched):
1140         logger.info(
1141             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
1142             floor.",
1143             stitching.min_shot_count,
1144             len(matched) - len(kept),
1145             len(matched),
1146         )
1147     if not kept:
1148         raise _CompileError(
1149             400,
1150             f"All {len(matched)} matching segments fall below "
1151             f"stitching.min_shot_count={stitching.min_shot_count}.",
1152         )
1153     return video, kept, out_name

```

32. user (2026-07-08T06:10:42.805Z)

```
28: _ENCODER_AVAILABLE: Dict[str, bool] = {}
31: def _probe_encoder_available(encoder: str) -> bool:
36:     if encoder in _ENCODER_AVAILABLE:
37:         return _ENCODER_AVAILABLE[encoder]
66:     _ENCODER_AVAILABLE[encoder] = ok
277: def _select_encoder(self) -> Tuple[List[str], str]:
458: def compile_highlights(
```

33. assistant (2026-07-08T06:12:05.277Z)

`min\_rally\_duration` isn't applied in the compiler and both paths share `\_resolve\_compile`, so no filter divergence. Now let me verify the worker's top-level exception handling (does a `cmd\_compile` crash still produce a failed-job signal?) and confirm the mirror-upload swallow behavior.

34. user (2026-07-08T06:12:09.064Z)

```
34:     load_config,
56:     except ValueError:
61: def _apply_overrides(config: Any, sets: List[str]) -> None:
170:     except StartupCheckError:
176:     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
179:     A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
183:     * rc 5 ? the execution has TERMINATED without a marker (`RemoteExecutionFailed`): a
worker
184:     crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of
the
198:     except RemoteExecutionFailed as e:
200:         "[worker] remote execution terminated without a completion marker: %s", e
203:     except PolicyTimeout as e:
205:         "[worker] remote compute did not complete (no marker within budget): %s", e
217: def _write_done_marker(
218:     done_uri: str,
226:     """M6: write a tiny JSON completion marker to `done_uri` (via the storage layer) for a
remote
229:         marker = {
237:             json.dump(marker, f)
238:             storage.put(local, done_uri, config=storage_cfg)
239:             logger.info("[worker] wrote completion marker -> %s", done_uri)
240:     except Exception as e: # never fail a good run because the marker push hiccuped
241:         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244: def cmd_infer(args: argparse.Namespace) -> int:
245:     config = load_config(args.config) if args.config else load_config()
307:     # mask the original failure. THEN the M6 done-marker (real rc + status) so a remote
319:     except Exception as e: # never mask the real failure on an artifact-push hiccup
321:     # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322:     if getattr(args, "done_uri", None):
323:         _write_done_marker(
324:             args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
411:     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
413:     # write their own marker via _fail() above, so the dispatcher fails FAST either way (no
30-min hang
414:     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local
run).
415:     if getattr(args, "done_uri", None):
416:         _write_done_marker(
417:             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
436: def _write_compile_marker(
437:     done_uri: str,
444:     """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri)
```



```

so the
447:         marker = {
457:             json.dump(marker, f)
458:             storage.put(local, done_uri, config=storage_cfg)
459:             logger.info("[worker] wrote compile marker -> %s", done_uri)
460:         except Exception as e: # never fail a good stitch because the marker push hiccuped
461:             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri, e)
469:     forked stitch pipeline), upload the reel to the bucket, and write a done-marker for the
dispatcher's
474:     config = load_config(args.config) if args.config else load_config()
518:     # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519:     if getattr(args, "done_uri", None):
520:         _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
587:     except (

```

35. assistant (2026-07-08T06:13:00.845Z)

Let me pin exact line numbers for the findings in the post-change files.

36. user (2026-07-08T06:13:06.336Z)

```

=== orch _stitch_reel mirror block ===
276:         "status": "success",
339:         return bool(report and report.get("status") == "success")
351:     the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
385:         if not report or report.get("status") != "success":
741:         "status": "success",
1014:         "status": "success",
1172:     reel_uri = reel_object_uri(cfg, video_id, name)
1190: def _stitch_reel(
1256:     reel_uri = reel_object_uri(cfg, video_id, out_name)
1264:     logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
=== cloud_run run_compile argv ===
6:     --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4 GPU).
74:     argv: List[str],
112:     argv: List[str],
254:     argv: List[str] = [
258:         "--out-uri",
260:         "--done-uri",
270:         argv += ["--set", kv]
307:     def run_compile(
323:         argv: List[str] = [
324:             "compile",
=== worker cmd_compile + parser ===
331:     status = "failed"
356:     status = "success"
464: def cmd_compile(args: argparse.Namespace) -> int:
513:     status = str(result.get("status") or "failed")
514:     rc = 0 if status == "success" else 3
520:     _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
809: def build_parser() -> argparse.ArgumentParser:
820:     sub = p.add_subparsers(dest="cmd", required=True)
822:     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
824:         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
828:         required=True,
865:     pc = sub.add_parser(
871:         required=True,
902:     pt = sub.add_parser(
908:         required=True,
914:         required=True,
917:     pt.add_argument("--version", required=True, help="semver for the artifact bundle")

```

37. assistant (2026-07-08T06:13:38.243Z)

I have enough to finalize. Let me verify the one remaining detail for Finding 1: that ``cmd_compile`` writes a success marker purely from ``_stitch_reel``'s returned status with no bucket-existence check.

38. user (2026-07-08T06:13:41.683Z)

```
495         ]
496         stitching = StitchingConfig(**(spec.get("stitching") or {}))
497
498         print(
499             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
500             f"encoder={stitching.encoder} out={out_name}"
501         )
502
503         result = orchestration._stitch_reel(
504             config,
505             stitching,
506             video_id,
507             source_ref,
508             time_pairs,
509             out_name,
510             locale,
511             lambda stage: print(f"[worker] compile: {stage}"),
512         )
513         status = str(result.get("status") or "failed")
514         rc = 0 if status == "success" else 3
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517
518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
519         # passes it.
520         if getattr(args, "done_uri", None):
521             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
522         return rc
```

39. assistant (2026-07-08T06:13:55.719Z)

I've completed my analysis. Working against the post-change tree at commit ``dac6736`` (the PR is not in local HEAD, so I read the merged-commit versions). Here are my test-gap findings.

Summary

The #521 tests are thorough on the config/placement resolver and the dispatch argv/marker shape, but they share one structural blind spot: **every test that reaches the reel stitch either stubs ``orchestration._stitch_reel`` (test_worker.py, test_compile_dispatch's default-path test) or replaces the worker with a fake ``CloudRunJobClient`` that writes the done-marker directly (test_compile_dispatch.py, test_compute_backend.py).** So the L4 worker's *real* stitch?mirror?marker path is never exercised. Two real failure modes hide behind that:

1. **Swallowed mirror-upload failure on the worker reports false success and loses the reel** (high ? silent data loss). ``_stitch_reel`` was written for the control-plane, where swallowing a mirror failure is safe because the local file survives and ``/api/highlights`` falls back to it. On the L4 worker the local reel is discarded when the Job exits (the code's own comment says the mirror "is how the reel reaches the bucket at all"), so swallowing ``storage.put`` failure and still returning ``status="success"`` yields an rc=0 success marker + a ``download_url`` that will 404 permanently. No test covers a worker-side mirror failure.
2. **The CP?worker contract (dispatch argv + spec JSON) is never round-tripped end-to-end** (medium). The dispatcher's emitted argv is asserted by a fake that never parses it, and the worker parser is tested with a hand-written argv; likewise the staged spec keys and the worker-read spec keys are two independent hand-built fixtures. A flag/key drift (e.g. ``--spec-uri`/`video_uri`/`time_pairs`` renamed on one side) passes all tests but makes the prod worker exit 2 (no marker) or mis-read the spec.

Let me report these.

40. user (2026-07-08T06:14:14.403Z)

Structured output provided successfully